

République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
École Supérieure Privée d'Ingénierie et de Technologies — ESPRIT

Rapport de Projet de Fin d'Études
Présenté en vue de l'obtention du Diplôme National d'Ingénieur en Informatique

MOUVE : Plateforme intelligente de compression et de transfert sécurisé de fichiers

Sous-titre : compression hybride assistée par IA, conteneur sécurisé et interfaces multi-canaux

Rubrique	Information
Réalisé par	TAIEB, Mohamed
Département / Filière	Informatique
Encadrant académique	À compléter
Encadrant industriel	À compléter
Entreprise d'accueil	Capgemini Engineering
Année universitaire	2025 - 2026

Document rédigé sur la base de la structure académique du rapport exemple fourni.

Autorisation de dépôt

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance. Cette page reprend l'organisation administrative du modèle fourni et conserve l'espace nécessaire à la validation de l'encadrant professionnel et de l'encadrant académique.

Le rapport concerne le projet MOUVE, réalisé par TAIEB Mohamed, et porte sur la conception d'une plateforme de compression intelligente et de transfert sécurisé de fichiers. Les signatures ci-dessous attestent que le document est prêt à être soumis pour évaluation académique.

Encadrant professionnel	Encadrant académique
Nom et prénom : Signature et cachet :	Nom et prénom : Signature :

Dédicaces

Je dédie ce travail à ma famille, dont le soutien moral et la confiance ont constitué une source constante de motivation durant toute la période du projet. Les efforts fournis dans ce rapport traduisent à la fois un engagement personnel et une volonté de valoriser les compétences acquises au cours de la formation d'ingénieur.

Je dédie également ce travail à mes enseignants, à mes collègues et à toutes les personnes qui ont contribué directement ou indirectement à la progression de ce projet. Leur accompagnement a permis de transformer une idée technique en une solution structurée, documentée et défendable dans un cadre académique et professionnel.

Remerciements

Mes remerciements s'adressent d'abord à l'établissement ESPRIT pour la qualité de la formation dispensée et pour l'environnement pédagogique qui a permis d'approfondir les compétences en développement logiciel, en ingénierie des systèmes et en intelligence artificielle.

Je remercie également l'entreprise Capgemini Engineering pour l'opportunité offerte et pour le cadre professionnel dans lequel ce projet a été développé. Les échanges techniques, les contraintes industrielles et les exigences de qualité observées au sein de l'entreprise ont fortement orienté la conception de la plateforme MOUVE.

Une reconnaissance particulière est adressée aux encadrants académique et industriel pour leurs conseils, leurs remarques et leur suivi. Leur accompagnement a contribué à préciser la problématique, à structurer la méthodologie de travail et à renforcer la cohérence entre les objectifs fonctionnels, l'architecture logicielle et les choix technologiques.

Table des matières

Introduction générale	9
Chapitre 1 — Contexte général	10
1.1 Organisme d'accueil	12
1.2 Problématique	13
1.3 Étude et critique de l'existant	14
1.4 Cahier des charges	15
1.5 Choix méthodologique	16
1.5.1 Application de la méthodologie PDCA	18
1.5.2 Diagramme de Gantt	19
Chapitre 2 — État de l'art	20
2.1 Systèmes de compression et de transfert sécurisés	22
2.2 Modèles et algorithmes de compression	27
2.3 Techniques d'optimisation	35
2.4 Compression apprise et représentation latente	36
2.5 Données de référence pour l'évaluation	41
2.6 Synthèse et justification des choix technologiques	47
Chapitre 3 — Conception et choix technologiques	54
3.1 Analyse des besoins	56
3.2 Architecture globale du système	60
3.3 Architectures partielles par étape	61
3.4 Choix technologiques	67
Chapitre 4 — Réalisation	82
4.1 Choix techniques	84
4.2 Plateformes pour l'exécution de MOUVE	85
4.3 Travail réalisé	91
4.4 Planning réel du projet	92
Conclusion générale	94
Références bibliographiques	95
Annexes	96

Table des figures

Figure 1.1 : Architecture générale de la plateforme MOUVE	12
Figure 1.2 : Cycle PDCA appliqué au projet MOUVE	16
Figure 1.3 : Diagramme de Gantt du projet MOUVE	19
Figure 2.1 : Architecture fonctionnelle d'une solution de compression sécurisée	25
Figure 2.2 : Pipeline de traitement des fichiers dans MOUVE	26
Figure 2.3 : Chaîne BWT, MTF, RLE et ZSTD	30
Figure 2.4 : Principe d'une compression apprise par représentation latente	36
Figure 2.5 : Réduction attendue selon la famille de fichiers	41
Figure 2.6 : Structure sécurisée du conteneur .nvc	49
Figure 3.1 : Architecture globale du système MOUVE	60
Figure 3.2 : Architecture partielle du moteur de compression classique	62
Figure 3.3 : Architecture partielle du mode appris	63
Figure 3.4 : Format sécurisé .nvc	66
Figure 3.5 : Architecture de déploiement cible	73
Figure 3.6 : Architecture de sécurité multicouche	76
Figure 4.1 : Diagramme de séquence de la compression via API	86
Figure 4.2 : Synthèse graphique des résultats attendus	91
Figure 4.3 : Planning réel et phases de réalisation	92
Figure annexe 2.1 : Déploiement cible de MOUVE	97

Liste des tableaux

Tableau 1.1 : Comparaison des solutions existantes 14

Tableau 1.2 : Synthèse du cahier des charges 15

Tableau 2.1 : Métriques d'évaluation de MOUVE 34

Tableau 2.2 : Récapitulatif des choix technologiques 53

Tableau 3.1 : Besoins fonctionnels de MOUVE 57

Tableau 3.2 : Principaux endpoints REST de MOUVE 65

Tableau 3.3 : Bibliothèques Python principales 80

Tableau 4.1 : Critères de sélection des plateformes 85

Tableau 4.2 : Comparaison des plateformes d'exécution 89

Liste des abréviations

La liste des abréviations regroupe les principaux termes techniques utilisés dans le rapport. Elle facilite la lecture des chapitres consacrés à la compression, à la sécurité, à l’architecture logicielle et à l’intelligence artificielle.

Abréviation	Signification
API	Application Programming Interface
BWT	Burrows-Wheeler Transform
MTF	Move-To-Front
RLE	Run-Length Encoding
ZSTD	Zstandard
JWT	JSON Web Token
RBAC	Role-Based Access Control
VQ-VAE	Vector Quantised Variational AutoEncoder
SOC2	System and Organization Controls 2
SHA-256	Secure Hash Algorithm 256 bits

Introduction générale

La croissance continue des volumes de données échangées par les systèmes informatiques modernes impose de nouvelles exigences en matière de stockage, de transfert et de sécurité. Les fichiers manipulés par les entreprises ne se limitent plus à des documents textuels simples ; ils comprennent des journaux applicatifs, des archives, des fichiers structurés, des binaires, des images et des jeux de données destinés à l'intelligence artificielle. Dans ce contexte, la compression de données demeure une problématique centrale, car elle permet de réduire l'espace de stockage, de diminuer les temps de transfert et d'optimiser les coûts d'infrastructure. Cependant, les approches classiques fondées sur un algorithme unique ne sont pas toujours adaptées à la diversité des formats et à la variabilité de leur entropie.

Le projet MOUVE s'inscrit dans cette problématique en proposant une plateforme de compression intelligente et de transfert sécurisé de fichiers. La solution combine un pipeline classique composé de la transformée de Burrows-Wheeler, du codage Move-To-Front, du Run-Length Encoding et de Zstandard, avec une stratégie de sélection automatique permettant de comparer le résultat du pipeline complet à une compression directe par Zstandard. Cette conception s'appuie sur les principes de la compression sans perte, où la réversibilité et l'intégrité sont indispensables, notamment lorsque les données concernent des fichiers professionnels ou techniques [1], [2], [3].

Au-delà du gain de taille, MOUVE intègre des mécanismes de sécurité destinés à contrôler les fichiers entrants, à assurer l'intégrité par hachage SHA-256, à protéger les conteneurs par chiffrement authentifié AES-GCM et à signer les métadonnées sensibles à l'aide d'Ed25519. Ces choix sont cohérents avec les recommandations de standards reconnus en cryptographie et en sécurité applicative [4], [5], [6], [7]. Le rapport comporte quatre chapitres. Le premier présente le contexte général du projet. Le deuxième expose l'état de l'art. Le troisième décrit la conception et les choix technologiques. Le quatrième détaille la réalisation, les plateformes logicielles et le planning réel.

Chapitre 1

Contexte général

Plan du chapitre :

- 1 Organisme d'accueil ;
- 2 Problématique ;
- 3 Étude et critique de l'existant ;
- 4 Cahier des charges ;
- 5 Choix méthodologique.

Ce chapitre reprend l'organisation du rapport exemple en introduisant le cadre industriel, la problématique générale, les limites des solutions existantes, les exigences du projet et la méthode de travail.

Chapitre 1. Contexte général

Introduction

Ce chapitre présente le cadre général dans lequel s'inscrit le projet MOUVE. Il commence par une présentation de l'organisme d'accueil, puis expose la problématique liée à la compression, à la sécurité et au transfert de fichiers. Une étude critique des solutions existantes est ensuite menée afin de positionner la contribution de la plateforme proposée. Le chapitre se poursuit par la définition du cahier des charges et se termine par la justification du choix méthodologique adopté pour conduire le projet.

1.1 Organisme d'accueil

Le projet a été réalisé dans un contexte professionnel lié à Capgemini Engineering, entité spécialisée dans les services d'ingénierie, le développement logiciel, la transformation numérique et l'accompagnement des projets industriels. L'environnement de travail favorise la mise en œuvre de solutions robustes, modulaires et documentées, ce qui correspond directement aux objectifs d'un projet de fin d'études en ingénierie informatique.

Dans ce cadre, la plateforme MOUVE a été pensée comme une solution logicielle capable de répondre à des contraintes proches de celles rencontrées en entreprise. Les besoins de performance, de sécurité, d'auditabilité et d'intégration multi-interface ont été considérés dès la phase d'analyse. Cette orientation professionnelle explique l'importance donnée à la traçabilité des fichiers, à l'authentification, au contrôle d'accès et au déploiement conteneurisé.

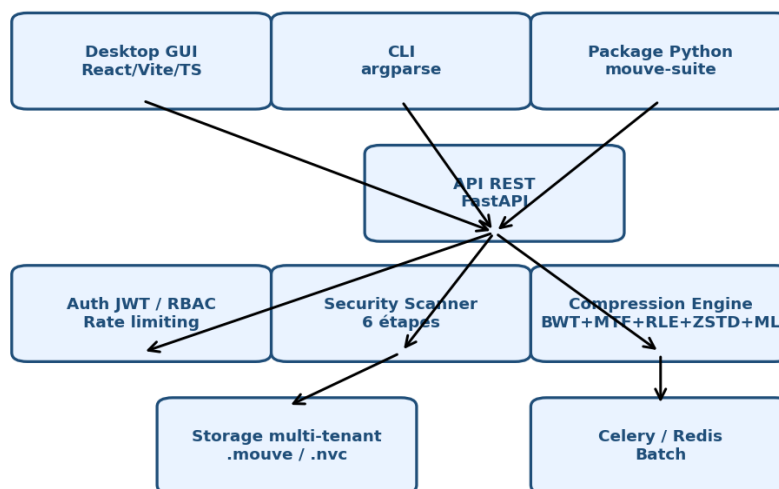


Figure 1.1 : Architecture générale de la plateforme MOUVE

1.2 Problématique

La compression de fichiers constitue une question ancienne mais toujours actuelle. Les algorithmes classiques comme DEFLATE, BZIP2 ou Zstandard fournissent des résultats efficaces dans de nombreux cas, mais leur performance dépend fortement de la structure interne des fichiers. Les données textuelles répétitives peuvent être fortement réduites, tandis que les données aléatoires ou déjà compressées résistent naturellement à la compression. Cette diversité rend insuffisante une stratégie uniforme appliquée à tous les fichiers [1], [2], [3].

La problématique principale consiste donc à concevoir une plateforme capable d'analyser les fichiers, de choisir automatiquement une approche de compression pertinente, d'éviter les surcoûts lorsque les données sont peu compressibles et de garantir une décompression exacte. À cette exigence de performance s'ajoute une contrainte de sécurité, car toute plateforme acceptant des fichiers utilisateurs est exposée à des risques de malwares, d'archives piégées, de contournement de type MIME ou de déni de service par fichiers volumineux [7].

La question de recherche peut être formulée ainsi : comment concevoir et réaliser une plateforme de compression sans perte, intelligente, sécurisée et intégrable, capable de fonctionner à travers une API REST, une interface en ligne de commande, une application Desktop et un package Python, tout en assurant la confidentialité, l'intégrité et l'auditabilité des fichiers traités ?

1.3 Étude et critique de l'existant

Les solutions existantes de compression se distinguent par leur compromis entre taux de compression, vitesse et compatibilité. Les formats ZIP et GZIP restent très répandus grâce à leur adoption historique, mais ils ne sont pas toujours optimaux pour les données structurées modernes. BZIP2 exploite la transformée de Burrows-Wheeler et fournit de bons taux sur les textes, mais son temps de traitement peut devenir contraignant pour des usages interactifs. Zstandard propose un excellent compromis entre vitesse et ratio, avec un format documenté par l'IETF et le support des dictionnaires [3].

Sur le plan du transfert sécurisé, les plateformes cloud offrent des fonctions de partage et d'authentification, mais elles ne donnent pas toujours un contrôle fin sur le pipeline de compression, la logique d'audit ou le format conteneur. Les solutions de sécurité applicative recommandent une défense en profondeur pour les uploads, incluant la validation du type, la limitation de taille, le stockage hors webroot, l'antivirus et l'analyse des archives [7]. MOUVE se positionne comme une solution intermédiaire entre les compresseurs classiques et les plateformes de transfert, en combinant compression, sécurité et traçabilité.

Tableau 1.1 : Comparaison des solutions existantes

Solution	Apport principal	Limite observée
ZIP / DEFLATE	Compatibilité élevée et usage universel	Ratio parfois inférieur sur données structurées
BZIP2	Bonne exploitation de la BWT pour les textes	Temps de compression plus élevé
Zstandard	Très bon compromis vitesse et ratio	Nécessite une stratégie d'adaptation
Plateformes cloud	Partage et stockage prêts à l'emploi	Contrôle limité du pipeline interne

1.4 Cahier des charges

Le cahier des charges de MOUVE définit une plateforme capable de compresser et de décompresser des fichiers sans perte, tout en assurant une expérience d'utilisation cohérente sur plusieurs interfaces. Le système doit accepter un fichier, l'analyser, exécuter un scan de sécurité, sélectionner une stratégie de compression, produire un conteneur exploitable et restituer le fichier original lors de la décompression.

Les exigences fonctionnelles comprennent la compression classique, la compression directe de comparaison, le mode appris basé sur un modèle PyTorch, la création de conteneurs .mouve et .nvc, l'exposition d'endpoints REST, l'intégration CLI, l'application Desktop et la publication sous forme de package Python. Les exigences non

fonctionnelles portent sur la sécurité, la maintenabilité, la scalabilité, l'intégrité cryptographique, l'isolation multi-tenant et la reproductibilité des traitements.

Tableau 1.2 : Synthèse du cahier des charges

Exigence	Description académique
Compression sans perte	La donnée restaurée doit être rigoureusement identique à l'original.
Sécurité des uploads	Le fichier doit être contrôlé avant toute opération intensive.
Intégrité	Un hash SHA-256 doit permettre de vérifier la conservation des données [4].
Confidentialité	Le conteneur sécurisé doit utiliser un mode de chiffrement authentifié [5].
Accessibilité	La solution doit être utilisable via API, CLI, Desktop et package Python.

1.5 Choix méthodologique

Afin d'organiser le développement du projet, la méthodologie PDCA a été retenue comme cadre d'amélioration continue. Cette approche correspond à la logique du rapport exemple et permet de structurer le travail autour de quatre phases complémentaires : planification, réalisation, vérification et amélioration. Dans un projet combinant algorithmes, sécurité et interfaces, cette démarche évite une progression linéaire rigide et favorise les ajustements progressifs.

La phase de planification a été consacrée à l'analyse de la problématique, à l'étude des solutions existantes et à la définition de l'architecture cible. La phase de réalisation a porté sur l'implémentation du moteur de compression, de l'API FastAPI, de la CLI, du Desktop et du package Python. La phase de vérification a concerné les tests d'intégrité, les scénarios de sécurité et les benchmarks. La phase d'amélioration a permis d'ajuster les paramètres de compression, les seuils de risque et l'organisation du rapport [15].

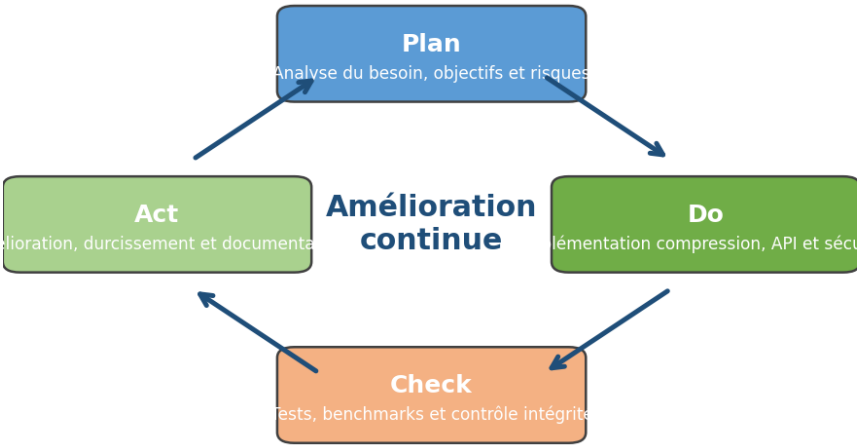


Figure 1.2 : Cycle PDCA appliqué au projet MOUVE

1.5.1 Application de la méthodologie PDCA

L'application concrète du cycle PDCA à MOUVE s'est traduite par des itérations successives sur le moteur, la sécurité et les interfaces. Pendant la phase Plan, les besoins ont été transformés en exigences vérifiables. Pendant la phase Do, les composants ont été développés de manière modulaire afin de permettre leur test indépendant. Pendant la phase Check, chaque résultat de compression a été comparé au fichier original à l'aide du hash SHA-256. Pendant la phase Act, les choix de niveau Zstandard, de taille de bloc et de politique de scan ont été ajustés selon les observations.

Cette méthodologie s'est révélée pertinente car la performance d'un compresseur dépend fortement des données. Une solution figée aurait risqué de privilégier un cas particulier au détriment d'autres formats. L'approche itérative

a permis de conserver une logique adaptable, dans laquelle la classification du fichier et la sélection stratégique constituent deux mécanismes essentiels de robustesse.

1.5.2 Diagramme de Gantt

Le diagramme de Gantt illustre la planification du projet de février à août. Les premières semaines ont été consacrées au cadrage et à l'état de l'art. Les mois suivants ont porté sur la conception, le développement du moteur, l'intégration de la sécurité, la réalisation des interfaces et la rédaction. La rédaction a été conduite en parallèle afin d'assurer la cohérence entre les choix techniques et leur justification académique.

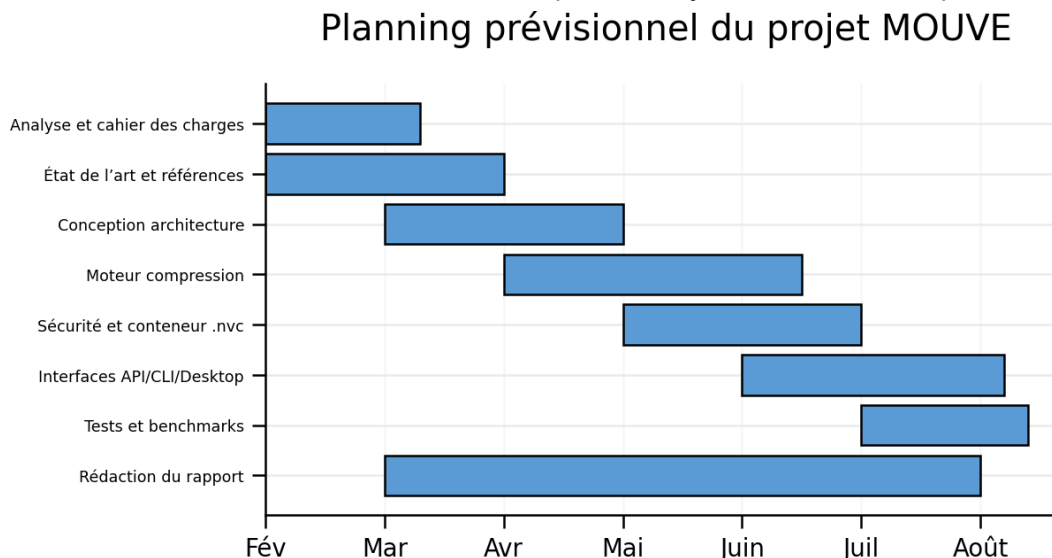


Figure 1.3 : Diagramme de Gantt du projet MOUVE

1.1.1 Positionnement de Capgemini Engineering dans l'ingénierie numérique

Capgemini Engineering se situe dans un espace professionnel où convergent ingénierie produit, développement logiciel, systèmes embarqués, cybersécurité, données et intelligence artificielle. Cette position impose une culture de livrables traçables, d'architectures maintenables et de processus reproductibles. Dans le cadre de MOUVE, l'intérêt de l'environnement d'accueil apparaît dans la transformation d'un problème algorithmique de compression en une plateforme complète comprenant des interfaces, des politiques de sécurité, un format de fichier et un scénario de déploiement.

Le contexte de service d'ingénierie implique également que la solution puisse être transférée d'un projet à un autre. La portabilité et la modularité deviennent donc des objectifs de conception, et non de simples qualités secondaires. La séparation entre moteur, API, sécurité et stockage répond à cette exigence en permettant de substituer un backend, un scanner ou une interface sans modifier l'ensemble du système.

Cette analyse montre que la valeur du projet ne réside pas uniquement dans un ratio de compression. Elle réside dans la capacité à intégrer une fonction technique au sein d'une chaîne exploitable, auditable et industrialisable. Le rapport doit ainsi évaluer MOUVE à la fois comme algorithme, composant logiciel et service sécurisé.

1.2.1 Enjeux industriels du stockage et des transferts

Dans les systèmes industriels, les volumes à transférer comprennent des journaux de diagnostic, des configurations, des archives de tests, des traces réseau et des datasets. La réduction de taille peut diminuer les coûts de stockage et réduire la durée d'échange, mais elle peut aussi augmenter la charge CPU et la latence. Le choix d'un pipeline doit donc être contextualisé par le coût global de traitement, la criticité des données et les contraintes de restitution.

Les environnements multi-tenant introduisent une difficulté supplémentaire. Deux utilisateurs peuvent soumettre simultanément des fichiers volumineux, ce qui crée un risque de contention sur la mémoire, le processeur et le stockage temporaire. Le système doit imposer des quotas, traiter les tâches longues de manière asynchrone et isoler les espaces de fichiers afin d'éviter les interférences et les fuites de données.

MOUVE traite ces enjeux par sa stratégie de sélection, son architecture Celery/Redis et ses contrôles d'entrée. L'intérêt académique consiste à montrer comment les propriétés algorithmiques sont traduites en décisions d'architecture et en critères mesurables.

1.3.1 Analyse critique des solutions commerciales et open source

Les outils ZIP, GZIP, BZIP2, LZMA et Zstandard sont matures, largement testés et disponibles sur de nombreuses plateformes. Une nouvelle solution ne peut donc être justifiée par la seule existence d'un pipeline différent. Elle doit démontrer une valeur d'intégration, une adaptabilité ou une gouvernance que ces outils ne fournissent pas directement. MOUVE se positionne comme un orchestrateur qui sélectionne, sécurise, encapsule et expose la compression à travers plusieurs interfaces.

Cette valeur ajoutée doit cependant être évaluée avec prudence. Un format propriétaire augmente le coût d'interopérabilité et de maintenance. Une chaîne BWT naïve peut être plus lente qu'un compresseur natif optimisé. Un scanner multicouche peut créer des faux positifs et augmenter la latence. La critique de l'existant conduit donc à considérer Zstandard comme baseline robuste et à conserver la possibilité de revenir au format direct.

La contribution du projet est recevable si la plateforme démontre une meilleure adéquation au besoin global, même lorsque son ratio brut n'est pas systématiquement supérieur. Cette distinction entre performance locale et valeur système est essentielle pour un mémoire d'ingénieur.

1.4.1 Traçabilité des exigences et critères d'acceptation

Le cahier des charges doit être transformé en exigences vérifiables. Une exigence telle que « compresser efficacement » est insuffisante tant qu'elle n'est pas associée à un corpus, une métrique, un seuil ou une méthode de comparaison. De même, « sécuriser les fichiers » doit être décliné en contrôles de taille, de type, d'archive, de malware, d'autorisation et d'intégrité.

La traçabilité relie chaque exigence à un composant, un test et une preuve. L'exigence de restitution sans perte est liée au moteur de décompression et vérifiée par SHA-256. L'isolation est liée au stockage tenant et vérifiée par des tests d'accès croisé. La résilience batch est liée à Celery/Redis et vérifiée par des scénarios d'indisponibilité et de reprise.

Cette matrice d'exigences constitue un outil de soutenance important, car elle permet de démontrer que la conception et les tests répondent effectivement aux objectifs initiaux.

Chapitre 1. Conclusion

Ce chapitre a permis de définir le cadre général du projet MOUVE. L'analyse de l'organisme d'accueil, de la problématique et de l'existant a montré que la compression ne peut pas être envisagée uniquement sous l'angle algorithmique. Elle doit également intégrer des exigences de sécurité, d'intégrité, d'expérience utilisateur et de déploiement. Ces éléments justifient l'étude technique approfondie présentée dans le chapitre suivant.

Chapitre 2

État de l'art

Plan du chapitre :

- 1 Systèmes de compression et transfert sécurisés ;
- 2 Modèles et algorithmes de compression ;
- 3 Techniques d'optimisation ;
- 4 Compression apprise ;
- 5 Jeux de données de référence ;
- 6 Synthèse et justification des choix technologiques.

Chapitre 2. État de l'art

Introduction

Ce chapitre présente les fondements scientifiques et techniques nécessaires à la compréhension de MOUVE. L'état de l'art commence par les systèmes de compression et de transfert sécurisé, puis examine les algorithmes de compression sans perte, les transformations préparatoires, les métriques d'évaluation, les techniques d'optimisation, la compression apprise et les jeux de données utilisés pour valider les performances. Les choix retenus sont ensuite justifiés à partir des contraintes du projet et des références reconnues.

2.1 Systèmes de compression et de transfert sécurisés : définition et architecture

2.1.5 Positionnement critique de MOUVE dans l'état de l'art

L'examen des solutions existantes montre que les compresseurs généralistes optimisent principalement le couple vitesse-ratio, tandis que les plateformes de transfert privilégient l'authentification, la disponibilité et la gouvernance. MOUVE se situe à l'intersection de ces deux familles. Cette position constitue un avantage architectural, mais également une source de complexité : toute optimisation du moteur doit être évaluée en tenant compte du coût du scanner, du conteneur, du stockage et de la vérification d'intégrité. La pertinence scientifique du projet ne repose donc pas sur l'affirmation d'une supériorité universelle, mais sur la capacité à sélectionner une stratégie adaptée et à expliciter le compromis obtenu [1]–[5].

La chaîne BWT–MTF–RLE est justifiée par sa capacité à transformer des dépendances contextuelles en séquences de faibles indices, mais elle introduit un coût calculatoire et des métadonnées supplémentaires. Zstandard, dont le format intègre des blocs, du codage entropique et des dictionnaires, constitue une référence plus robuste pour les faibles latences [3], [4]. Le choix de MOUVE consiste dès lors à ne pas opposer ces approches : la compression directe joue le rôle de baseline interne, tandis que le pipeline transformé est retenu uniquement lorsque son gain compense son surcoût. Cette stratégie réduit le risque de dégradation sur les données déjà compressées ou à entropie élevée.

Un système de compression et de transfert sécurisé désigne une chaîne logicielle capable de réduire la taille d'un fichier tout en préservant sa possibilité de reconstruction exacte et en protégeant les données contre les altérations ou les accès non autorisés. La compression sans perte impose que la sortie décompressée soit identique à l'entrée, ce qui distingue ce domaine de la compression avec perte utilisée pour l'audio ou l'image.

Dans une architecture professionnelle, la compression est rarement isolée. Elle est associée à l'authentification, au contrôle d'accès, au stockage, à la journalisation, au chiffrement et à la supervision. MOUVE reprend cette vision systémique en plaçant le moteur de compression au centre d'une plateforme accessible par plusieurs interfaces et entourée de mécanismes de sécurité.

2.1.1 Définition et enjeux

Les enjeux de la compression moderne dépassent la réduction de taille. Le système doit préserver l'intégrité du fichier, limiter le coût de calcul, s'adapter à des formats variés et rester compatible avec des chaînes de traitement automatisées. Zstandard est représentatif de cette évolution car il combine rapidité, niveau de compression ajustable et prise en charge des dictionnaires [3].

La sécurité constitue un second enjeu majeur. Une plateforme de transfert recevant des fichiers peut devenir un vecteur d'attaque si elle ne vérifie pas les extensions, les types réels, les tailles, les archives et les contenus potentiellement malveillants. Les recommandations OWASP relatives aux uploads soulignent la nécessité d'une défense en profondeur plutôt que d'un contrôle unique [7].

2.1.2 Architecture des solutions de compression sécurisée

Une architecture de compression sécurisée comprend généralement une couche d'interface, une couche d'authentification, un module d'analyse des fichiers, un moteur de compression, un format conteneur, un stockage et une couche d'audit. L'API joue un rôle d'orchestrateur en imposant les politiques de sécurité avant de transmettre les données au moteur.

Dans MOUVE, l'architecture est organisée de manière modulaire afin que chaque responsabilité soit isolée. Cette organisation facilite l'évolution du système, car il devient possible de remplacer un classificateur, un backend de stockage ou un modèle de compression appris sans modifier l'ensemble de la plateforme.

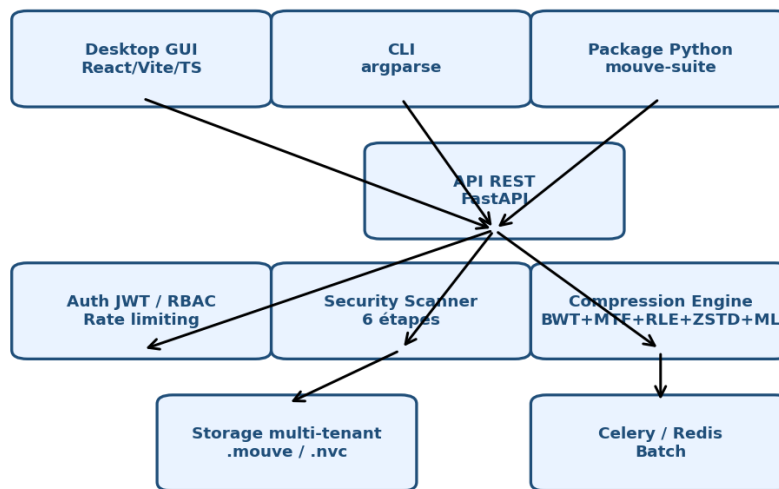


Figure 2.1 : Architecture fonctionnelle d'une solution de compression sécurisée

2.1.3 Fonctions principales

Les fonctions principales attendues d'une plateforme comme MOUVE sont la compression, la décompression, la classification, l'analyse de sécurité, la création de conteneurs, l'authentification, la gestion de l'historique et la restitution de résultats vérifiables. Ces fonctions doivent former une chaîne cohérente dans laquelle chaque étape produit des métadonnées utiles aux étapes ultérieures.

Le principe de fonctionnement est le suivant : le fichier est reçu par une interface, il est contrôlé par le scanner, il est classifié selon des critères statistiques, il est compressé par la stratégie la plus pertinente, puis il est encapsulé avec ses métadonnées. Lors de la décompression, ces métadonnées guident l'inversion exacte du pipeline.

2.1.4 Pipeline de traitement des fichiers

Le pipeline de traitement de MOUVE s'inspire des architectures de compression par blocs et des pipelines de prétraitement. Il commence par une analyse rapide du fichier, puis applique les transformations nécessaires avant le codage final. L'approche choisie repose sur l'idée que certaines transformations rendent les données plus favorables au codage entropique [1], [2].

Le pipeline n'est pas appliqué aveuglément. La sortie issue de la chaîne complète est comparée à une compression directe par Zstandard. Cette comparaison limite les effets négatifs sur les fichiers incompressibles et améliore la robustesse de la solution dans un environnement hétérogène.



Stratégie intelligente : comparaison du pipeline complet avec ZSTD direct puis conservation du résultat le plus compact.

Figure 2.2 : Pipeline de traitement des fichiers dans MOUVE

2.2 Modèles et algorithmes de compression

Les algorithmes de compression sans perte cherchent à représenter une information de manière plus compacte sans supprimer d'information. Les familles les plus courantes reposent sur la substitution par dictionnaire, le codage statistique ou les transformations qui réorganisent les données avant compression. La performance dépend alors de la redondance présente dans le fichier et de l'aptitude de l'algorithme à l'exploiter.

MOUVE adopte une approche hybride. La chaîne BWT, MTF et RLE prépare les données en créant des séquences plus régulières. Zstandard effectue ensuite le codage final. Cette architecture s'inspire des compresseurs basés sur la transformée de Burrows-Wheeler tout en bénéficiant des performances d'un codec moderne documenté par l'IETF [1], [2], [3].

2.2.1 Approches statistiques et dictionnaires

Les approches par dictionnaire, comme celles issues de la famille LZ, remplacent des séquences répétées par des références plus courtes. Elles sont efficaces lorsque les motifs apparaissent plusieurs fois dans un même fichier ou au sein d'un corpus similaire. Les dictionnaires entraînés améliorent cette logique en capturant les motifs représentatifs d'un ensemble de fichiers.

Zstandard combine une recherche de correspondances avec des méthodes de codage entropique et permet l'utilisation de dictionnaires. Cette caractéristique est importante pour MOUVE, car elle ouvre la voie à des profils spécialisés pour des familles de fichiers comme les logs, les JSON, les CSV ou les exports applicatifs [3].

2.2.2 Transformées, codage entropique et pipelines hybrides

Les transformées ne compressent pas nécessairement par elles-mêmes. Leur rôle est de modifier l'organisation des symboles afin de rendre les étapes suivantes plus efficaces. La transformée de Burrows-Wheeler est particulièrement connue pour rapprocher les symboles similaires et créer des runs exploitables par MTF et RLE [1], [2].

Le codage entropique intervient ensuite pour représenter les symboles fréquents avec moins de bits. Dans MOUVE, cette étape est confiée à Zstandard afin de bénéficier d'un codec robuste, rapide et largement documenté. Le caractère hybride du pipeline réside dans l'association d'une préparation structurelle et d'un codage final moderne.

2.2.3 La chaîne BWT, MTF, RLE et ZSTD

La chaîne adoptée par MOUVE commence par BWT, qui produit une permutation réversible du bloc. Cette permutation a pour objectif de regrouper les symboles ayant des contextes proches. MTF transforme ensuite les symboles en indices selon leur récurrence, ce qui produit fréquemment de petites valeurs répétées. RLE encode ces répétitions, puis Zstandard finalise la compression.

Cette chaîne est pertinente sur les fichiers textuels, structurés ou répétitifs. Elle peut toutefois introduire un overhead sur les données déjà compressées ou aléatoires. C'est pourquoi MOUVE conserve une stratégie de sélection automatique qui compare le pipeline complet au mode direct.



Stratégie intelligente : comparaison du pipeline complet avec ZSTD direct puis conservation du résultat le plus compact.

Figure 2.3 : Chaîne BWT, MTF, RLE et ZSTD

2.2.3.1 Principe de fonctionnement

Le principe de fonctionnement repose sur la séparation entre organisation de l'information et codage final. BWT expose des régularités de contexte. MTF transforme ces régularités en petites valeurs. RLE réduit les séquences répétées. Zstandard exploite ensuite une représentation plus favorable qu'un flux brut non préparé. Cette logique est cohérente avec les analyses montrant l'intérêt de la BWT comme transformation préparatoire [2].

2.2.3.2 Formalisation mathématique

Pour un bloc de taille n , la transformée de Burrows-Wheeler peut être exprimée comme une permutation réversible des symboles obtenue après tri lexicographique des rotations ou des suffixes. La décompression nécessite l'indice BWT ou un marqueur de fin permettant de reconstruire l'ordre original par la correspondance Last-First. La réversibilité impose que les métadonnées associées au bloc soient conservées avec précision [1].

Le gain de compression attendu dépend de la distribution de symboles après transformation. Lorsque la BWT produit de longues séquences de symboles similaires, MTF et RLE deviennent très efficaces. Lorsque la distribution reste uniforme, la compression reste limitée, ce qui justifie l'usage d'un classificateur et d'une stratégie de repli.

```
BWT(data) -> (last_column, primary_index)
MTF(last_column) -> index_stream
RLE(index_stream) -> run_stream
ZSTD(run_stream) -> payload
```

2.2.3.3 Évolution des approches hybrides

Les approches hybrides répondent à une intuition forte : aucun algorithme unique n'est optimal pour tous les formats. Les compresseurs modernes cherchent donc à adapter leurs paramètres, à utiliser des dictionnaires, à changer de niveau de compression ou à détecter les données incompressibles. MOUVE applique ce raisonnement en comparant explicitement deux stratégies et en conservant le résultat le plus compact.

Cette logique d'adaptation est également compatible avec l'intégration de l'apprentissage automatique. Un modèle peut apprendre des profils de données spécifiques et proposer une représentation latente compacte, tandis que le pipeline classique reste disponible comme base fiable et explicable.

2.2.4 Métriques d'évaluation

L'évaluation d'un compresseur ne peut pas se limiter au ratio. Il faut considérer le temps de compression, le temps de décompression, l'utilisation mémoire, le coût CPU, la robustesse et l'intégrité du fichier restauré. Pour une plateforme de transfert, la sécurité et la capacité d'audit font également partie des critères d'évaluation.

Dans MOUVE, le ratio est mesuré par la taille compressée divisée par la taille originale. Le gain est obtenu par un moins ce ratio. L'intégrité est vérifiée par comparaison SHA-256 entre l'original et la donnée restaurée, conformément au rôle du hachage dans la détection des modifications [4].

Tableau 2.1 : Métriques d'évaluation de MOUVE

Métrique	Définition
Ratio	Taille compressée divisée par taille originale.
Gain	Réduction relative obtenue après compression.
Latence	Temps nécessaire pour compresser ou décompresser.
Mémoire	Quantité de RAM mobilisée pendant le traitement.
Intégrité	Égalité entre le hash original et le hash restauré.

2.3 Techniques d'optimisation

2.3.1 Critères de justification technologique

Chaque technologie retenue doit être évaluée selon des critères reproductibles : maturité, interopérabilité, coût de calcul, sécurité, maintenabilité et capacité à être testée. FastAPI est pertinent parce qu'il s'appuie sur OpenAPI et les annotations de type, ce qui améliore la traçabilité contractuelle des endpoints [20], [21]. React et TypeScript sont cohérents avec une interface riche et typée, tandis que Vite fournit un cycle de développement rapide et un build optimisé [22], [23]. Celery et Redis répondent au besoin d'isoler les traitements longs du cycle HTTP, mais exigent une gestion rigoureuse des retries, des timeouts, de l'idempotence et de la visibilité des tâches [24], [25].

Sur le plan de la qualité, l'évaluation est alignée sur les caractéristiques du modèle ISO/IEC 25010: adéquation fonctionnelle, efficacité des performances, fiabilité, sécurité, maintenabilité et portabilité [31]. Cette référence évite de réduire la validation à une démonstration fonctionnelle. Elle impose au contraire de discuter les erreurs, les limites de capacité, les consommations mémoire, les risques de sécurité et l'effort de maintenance.

Les optimisations concernent à la fois les algorithmes et l'architecture logicielle. Au niveau du moteur, l'utilisation de dictionnaires Zstandard, la taille des blocs, le niveau de compression et la sélection directe ou complète influencent fortement les performances. Au niveau système, les traitements asynchrones, le stockage temporaire maîtrisé et l'isolation des utilisateurs contribuent à la stabilité.

L'optimisation ne doit pas compromettre la décompression sans perte. Toute transformation doit être associée à des métadonnées suffisantes pour être inversée. De plus, l'optimisation ne peut pas réduire la sécurité ; les contrôles d'upload, le chiffrement authentifié et les journaux d'audit doivent rester actifs même lorsque le système vise un débit élevé [5], [7], [17].

2.4 Compression apprise et représentation latente

2.4.5 Limites scientifiques de la compression apprise

La littérature sur la compression neuronale est majoritairement orientée vers les images, la vidéo ou l'audio et optimise un compromis débit-distorsion. Or, MOUVE cible des fichiers génériques et revendique une restitution sans perte. Cette différence est fondamentale : une reconstruction visuellement satisfaisante n'est pas acceptable lorsque le fichier restauré doit être identique bit à bit. Les travaux sur les VQ-VAE démontrent l'intérêt des représentations discrètes [26], tandis que les architectures récentes de compression par VAE montrent la nécessité d'un modèle probabiliste et d'une quantification soigneusement conçue [27]. Dans MOUVE, le mode appris doit donc rester expérimental tant qu'une preuve de réversibilité complète n'est pas apportée.

L'analyse critique conduit à recommander deux voies. La première consiste à employer l'apprentissage pour sélectionner les paramètres d'un compresseur classique sans modifier les données, ce qui préserve naturellement le caractère sans perte. La seconde consiste à utiliser un modèle appris pour produire des prédictions ou des contextes ensuite codés par un mécanisme entropique réversible. Dans les deux cas, le protocole doit comparer le gain net, incluant le poids du modèle, le temps d'inférence et le coût de versionnement.

La compression apprise explore l'idée qu'un modèle entraîné sur un corpus spécifique peut représenter les données dans un espace latent plus compact. Les autoencodeurs et les VQ-VAE constituent des approches pertinentes car ils apprennent des codes latents, continus ou discrets, susceptibles de capturer les structures récurrentes des données [11].

Dans MOUVE, le mode appris reste optionnel et conditionné à la disponibilité d'un modèle entraîné. Cette prudence est nécessaire car un modèle mal entraîné peut produire des représentations inefficaces ou coûteuses. Le pipeline classique demeure donc le mode principal, tandis que le mode appris est envisagé comme une extension spécialisée pour des corpus homogènes.



Le mode appris est réservé aux familles de fichiers répétitives et évalué par benchmark avant activation.

Figure 2.4 : Principe d'une compression apprise par représentation latente

2.4.1 Motivations et principe

La motivation du mode appris réside dans les limites des algorithmes génériques face à des données très spécifiques. Un corpus de logs industriels, de traces réseau ou de fichiers applicatifs peut contenir des motifs propres à un domaine. Un modèle entraîné sur ce corpus peut apprendre une représentation plus compacte que celle obtenue par un compresseur généraliste.

Le principe consiste à transformer un bloc de données en un code latent, puis à compresser ce code par Zstandard. La reconstruction dépend du modèle et des paramètres enregistrés. L'usage en production exige donc une gestion stricte des versions de modèles et une vérification systématique de l'intégrité après décodage.

2.4.2 Formulation mathématique

Un autoencodeur peut être décrit par une fonction d'encodage f_θ qui transforme une entrée x en représentation z , et par une fonction de décodage g_ϕ qui reconstruit x à partir de z . L'objectif d'entraînement minimise généralement une erreur de reconstruction, à laquelle peuvent être ajoutées des contraintes de quantification ou de régularisation. Dans un VQ-VAE, la représentation latente est discrétisée par vector quantization, ce qui rend le code plus proche d'une séquence compressible [11].

Pour une compression sans perte stricte, l'usage d'un autoencodeur doit être encadré, car les modèles neuronaux classiques sont souvent associés à des reconstructions approximatives. MOUVE traite donc le mode appris comme une stratégie expérimentale destinée à produire une représentation intermédiaire compressible, tout en conservant des contrôles d'intégrité forts.

```
z = f_theta(x)
q = quantize(z)
payload = ZSTD(q)
x_hat = g_phi(decompress(payload))
validation = SHA256(x) == SHA256(x_hat)
```

2.4.3 Taxonomie des méthodes de compression apprises

Les méthodes de compression apprises peuvent être classées selon la nature de leur représentation. Les autoencodeurs continus apprennent une représentation dense, tandis que les VQ-VAE apprennent une représentation discrète. Les modèles génératifs peuvent améliorer la qualité de reconstruction dans les domaines multimédias, mais leur intérêt pour la compression sans perte de fichiers génériques reste limité par les exigences de réversibilité.

Pour MOUVE, la priorité est donnée à un mode appris pragmatique, utilisable sur des données dont la structure est stable et contrôlée. Cette décision évite de généraliser abusivement les résultats obtenus sur un corpus particulier. Elle maintient également l'explicabilité du pipeline classique.

2.4.4 Compression sécurisée et intégrité

La compression et la sécurité doivent être considérées conjointement. Un conteneur compressé peut être altéré, substitué ou rejoué. MOUVE répond à ce risque par des métadonnées de hash, un chiffrement authentifié AES-GCM et une signature Ed25519. AES-GCM fournit à la fois confidentialité et authentification du message dans le cadre défini par NIST SP 800-38D [5]. Ed25519 est décrit par le RFC 8032 comme une instance d'EdDSA fondée sur les courbes Edwards [6].

L'intégrité repose sur SHA-256, dont le standard FIPS 180-4 précise l'usage pour générer des empreintes de messages et détecter les modifications [4]. Dans MOUVE, ces mécanismes ne remplacent pas le scanner de sécurité ; ils garantissent que les données acceptées, compressées et éventuellement transférées n'ont pas été modifiées sans détection.

2.5 Données de référence pour l'évaluation

L'évaluation de MOUVE nécessite plusieurs familles de fichiers afin de mesurer le comportement du pipeline dans des conditions variées. Un corpus uniquement textuel donnerait une vision trop favorable, tandis qu'un corpus

aléatoire masquerait les bénéfices du pipeline. Les catégories retenues couvrent donc les textes répétitifs, les données structurées, les binaires, les images, les archives et les données aléatoires.

Chaque famille présente une signature statistique différente. Les fichiers textuels ont souvent une faible entropie locale. Les JSON et CSV contiennent des clés et séparateurs répétitifs. Les images peuvent déjà être compressées. Les archives combinent plusieurs fichiers, parfois eux-mêmes compressés. Les données aléatoires permettent de vérifier la capacité du système à éviter une expansion inutile.

Ratio de compression attendu par famille de fichiers

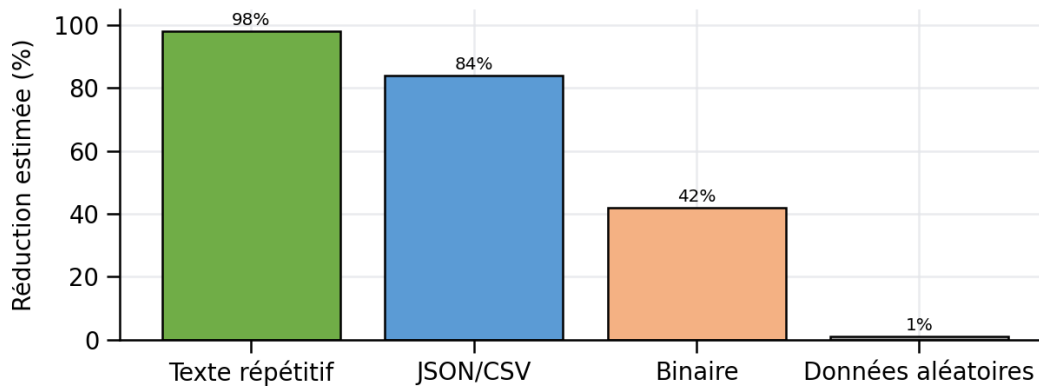


Figure 2.5 : Réduction attendue selon la famille de fichiers

2.5.1 Textes et journaux applicatifs

Les textes et journaux applicatifs constituent un cas favorable à la compression. Ils contiennent des mots, des motifs, des horodatages et des structures répétitives. La BWT peut regrouper les contextes semblables, MTF transforme cette régularité en indices faibles et RLE réduit les répétitions. Ce type de fichier permet de valider le potentiel maximal de MOUVE.

2.5.2 Données structurées JSON et CSV

Les fichiers JSON et CSV sont particulièrement intéressants car ils contiennent une structure syntaxique stable. Les clés JSON, les séparateurs, les noms de colonnes et les valeurs récurrentes produisent une redondance exploitable. L'usage de dictionnaires Zstandard peut améliorer davantage les résultats lorsque les fichiers appartiennent à une même application [3].

2.5.3 Binaires, images et médias

Les fichiers binaires et les images présentent un comportement plus variable. Certains binaires contiennent des zones répétitives, tandis que les images JPEG ou PNG sont déjà compressées. Dans ces cas, le pipeline complet peut devenir moins efficace que Zstandard direct. La stratégie de sélection automatique devient donc essentielle pour éviter une augmentation de taille.

2.5.4 Archives et fichiers composites

Les archives exigent une attention particulière car elles peuvent contenir une diversité de fichiers et présenter des risques de sécurité. Les bombes ZIP, les chemins relatifs malveillants et les archives imbriquées sont des menaces connues dans les mécanismes d'upload [7]. MOUVE doit analyser ces fichiers avant toute extraction ou traitement approfondi.

2.5.5 Données aléatoires et incompressibles

Les données aléatoires représentent un cas limite. Leur entropie élevée réduit fortement les possibilités de compression sans perte. Un bon système doit identifier ce comportement et limiter l'overhead. Le classificateur de MOUVE utilise l'entropie de Shannon et le ratio de caractères imprimables pour alimenter cette décision.

2.6 Synthèse et justification des choix technologiques

L'état de l'art conduit à justifier une architecture hybride. La BWT et les transformations associées apportent un gain sur les données textuelles et structurées. Zstandard fournit un backend rapide et documenté. Les mécanismes cryptographiques standards protègent l'intégrité et la confidentialité. L'API FastAPI facilite l'intégration tout en reposant sur OpenAPI et les annotations de type Python [8], [9].

2.6.1 Choix du pipeline classique

Le pipeline classique BWT, MTF, RLE et ZSTD est retenu car il combine préparation structurelle et compression efficace. Il est particulièrement adapté aux données répétitives et conserve une logique explicable, ce qui facilite la validation académique et la maintenance. La comparaison avec Zstandard direct constitue une protection contre les cas défavorables [1], [2], [3].

2.6.2 Choix du mode appris

Le mode appris est retenu comme extension et non comme base unique. Les VQ-VAE montrent l'intérêt des représentations discrètes apprises pour certaines données, mais l'usage dans une compression sans perte de fichiers génériques exige des vérifications strictes [11]. Cette prudence permet de concilier innovation et fiabilité.

2.6.3 Stratégie de chiffrement et signature

La stratégie de sécurité du format .nvc repose sur trois niveaux complémentaires. SHA-256 assure la détection de modification. AES-256-GCM assure la confidentialité et l'authenticité du contenu. Ed25519 permet de signer les métadonnées ou le hash global afin de vérifier l'origine du conteneur [4], [5], [6].



Format .nvc : chiffrement AES-256-GCM, signature Ed25519 et hachage global pour l'intégrité.

Figure 2.6 : Structure sécurisée du conteneur .nvc

2.6.4 Choix des jeux de données de validation

Les jeux de validation doivent couvrir les cas favorables et défavorables. Le corpus retenu combine textes répétitifs, fichiers JSON et CSV, binaires, images, archives et données aléatoires. Cette diversité évite de présenter un résultat artificiellement optimiste et permet d'évaluer la stratégie de sélection intelligente.

2.6.5 Récapitulatif des choix

Les choix technologiques retenus reposent sur un équilibre entre performance, fiabilité, sécurité et intégration. Le tableau suivant synthétise les décisions principales et leur justification dans le contexte du projet.

Tableau 2.2 : Récapitulatif des choix technologiques

Composant	Choix retenu	Justification
Compression	BWT, MTF, RLE, ZSTD	Approche hybride robuste et explicable.
Sécurité	SHA-256, AES-GCM, Ed25519	Standards reconnus pour intégrité, confidentialité et signature.
API	FastAPI et OpenAPI	Développement rapide et documentation automatique.
IA	PyTorch et VQ-VAE optionnel	Expérimentation sur corpus spécialisés.
Déploiement	Docker, Celery et Redis	Portabilité et traitements asynchrones.

2.2.5 Comparaison scientifique de ZIP, GZIP, BZIP2, LZMA et ZSTD

ZIP est un conteneur pouvant employer plusieurs méthodes, dont DEFLATE, tandis que GZIP encapsule généralement un seul flux DEFLATE. Ces solutions présentent une compatibilité élevée et une vitesse correcte, mais leur compromis ratio–vitesse est moins favorable que celui de codecs plus récents sur de nombreux corpus. BZIP2 applique BWT, MTF, RLE et codage de Huffman; il obtient souvent de bons ratios sur les textes, au prix d'un temps de compression plus important.

LZMA emploie un dictionnaire de grande taille, une modélisation avancée et du range coding. Il vise des ratios élevés mais demande davantage de mémoire et de temps. Zstandard combine des séquences de type LZ avec FSE et Huffman, propose de nombreux niveaux et conserve une décompression rapide. Le choix dépend donc de la taille, de la latence, de la mémoire, de la compatibilité et de la fréquence de décompression.

MOUVE ne doit pas être présenté comme un substitut universel. Sa contribution est la sélection et l'intégration sécurisée. La comparaison expérimentale doit mesurer la taille finale, y compris les en-têtes, et présenter les résultats par famille de données.

2.2.6 Entropie, redondance et limites théoriques

La théorie de Shannon relie la compressibilité à l'entropie de la source. Une distribution uniforme et indépendante contient peu de redondance exploitable, tandis qu'une source présentant des dépendances contextuelles peut être représentée plus efficacement. Cette limite explique pourquoi les données chiffrées, aléatoires ou déjà compressées ne doivent pas être soumises aveuglément à une chaîne coûteuse.

La BWT n'abaisse pas l'entropie du message; elle réordonne les symboles afin de rendre les contextes locaux plus accessibles à MTF, RLE et au codage entropique. La distinction entre transformation et compression doit être explicitée, car elle évite d'attribuer à la BWT un gain qui provient de la chaîne complète.

Dans MOUVE, l'entropie de Shannon constitue un indicateur de classification, mais elle ne suffit pas à prédire le résultat. La décision finale doit rester empirique, fondée sur la comparaison des sorties candidates.

2.3.2 Dictionnaires Zstandard et compression de petits objets

Les petits fichiers disposent de peu d'historique interne, ce qui limite l'apprentissage des motifs pendant la compression. Un dictionnaire entraîné sur des échantillons représentatifs fournit un contexte initial et peut améliorer le ratio sur des messages similaires. Cette technique est pertinente pour les JSON, événements et métadonnées produits par une même application.

Le dictionnaire devient toutefois un artefact versionné. La décompression nécessite le dictionnaire compatible, et son identifiant doit être enregistré dans le conteneur. Une politique de cycle de vie est nécessaire pour conserver les anciennes versions, mesurer la dérive du corpus et retirer les dictionnaires uniquement lorsqu'aucun fichier actif n'en dépend.

MOUVE peut intégrer cette technique comme profil spécialisé. Le gain doit inclure le coût de stockage et de distribution du dictionnaire, particulièrement lorsque les fichiers sont peu nombreux.

2.4.6 Comparaison des approches IA de compression

Les approches apprises peuvent être classées en modèles auto-régressifs, autoencodeurs variationnels, flows et modèles à quantification vectorielle. Les modèles auto-régressifs estiment précisément les distributions conditionnelles mais peuvent être lents au décodage. Les VAE optimisent une représentation latente et un modèle entropique; ils sont dominants en compression multimédia avec perte. Les flows offrent une bijection exacte mais leur coût et leur architecture peuvent limiter l'usage.

Le VQ-VAE produit des codes discrets associés à un codebook, ce qui facilite le codage mais ne garantit pas une reconstruction bit-à-bit pour des fichiers génériques. Pour MOUVE, l'IA est plus naturellement utilisée pour sélectionner une stratégie ou prédire des paramètres que pour remplacer directement un compresseur sans perte.

La comparaison doit inclure le coût du modèle, la taille du codebook, la latence, la reproductibilité, la compatibilité des versions et la robustesse aux données hors distribution.

2.4.7 Comparaison des mécanismes de sécurité

SHA-256 fournit une empreinte d'intégrité mais ne protège pas la confidentialité et ne prouve pas l'identité de l'émetteur. HMAC ajoute une authentification symétrique fondée sur un secret partagé. AES-GCM assure simultanément chiffrement et authentification, à condition que le nonce soit unique pour une même clé. Ed25519 fournit une signature asymétrique vérifiable sans partager la clé privée.

Ces mécanismes sont complémentaires. Dans .nvc, AES-GCM protège le payload, SHA-256 facilite la vérification et l'identification, et Ed25519 peut signer le hash ou les métadonnées. La conception doit définir précisément les données associées authentifiées, la gestion des nonces, la rotation des clés et le comportement en cas d'échec de vérification.

La sécurité du format dépend davantage de la gestion des clés que du choix des primitives. Les clés ne doivent pas être intégrées au code ou aux images Docker; elles doivent provenir d'un secret manager ou d'un mécanisme équivalent.

Chapitre 2. Conclusion

Ce chapitre a présenté les fondements théoriques et techniques de MOUVE. L'analyse a montré que la solution devait associer un pipeline classique éprouvé, un backend moderne, une stratégie de sélection automatique, un mécanisme de sécurité fondé sur des standards et une architecture logicielle intégrable. Ces choix guident la conception détaillée du chapitre suivant.

Chapitre 3

Conception et choix technologiques

Plan du chapitre : 1 Analyse des besoins ; 2 Architecture globale du système ; 3 Architectures partielles par étape ; 4 Choix technologiques. Le chapitre décrit la conception de MOUVE en suivant la même organisation que le rapport exemple.

Chapitre 3. Conception et choix technologiques

Introduction

Ce chapitre est consacré à la conception de la plateforme MOUVE. Il présente l'analyse des besoins, l'architecture globale, les architectures partielles correspondant aux différents modules et les choix matériels et logiciels retenus. L'objectif est de montrer comment les exigences définies dans le cahier des charges ont été transformées en une architecture cohérente, modulaire et vérifiable.

3.1 Analyse des besoins

3.1.4 Modélisation UML des besoins

La modélisation UML complète l'analyse textuelle en représentant les acteurs, les responsabilités et les dépendances. Le diagramme de cas d'utilisation formalise les objectifs externes sans exposer les détails d'implémentation. Il montre que l'utilisateur interagit avec la compression, la décompression et l'historique, que l'administrateur contrôle les utilisateurs et les audits, et qu'un service externe consomme l'API ou soumet des traitements batch. La frontière du système clarifie ce qui relève de MOUVE et ce qui dépend de son environnement.

Cette représentation met également en évidence deux préoccupations transverses. Premièrement, l'authentification précède toutes les opérations sensibles. Deuxièmement, le scan de sécurité est inclus dans les cas d'utilisation de compression et de décompression plutôt que proposé comme une option isolée. Ce choix traduit une approche security-by-design cohérente avec les recommandations OWASP sur les uploads [11], [12].

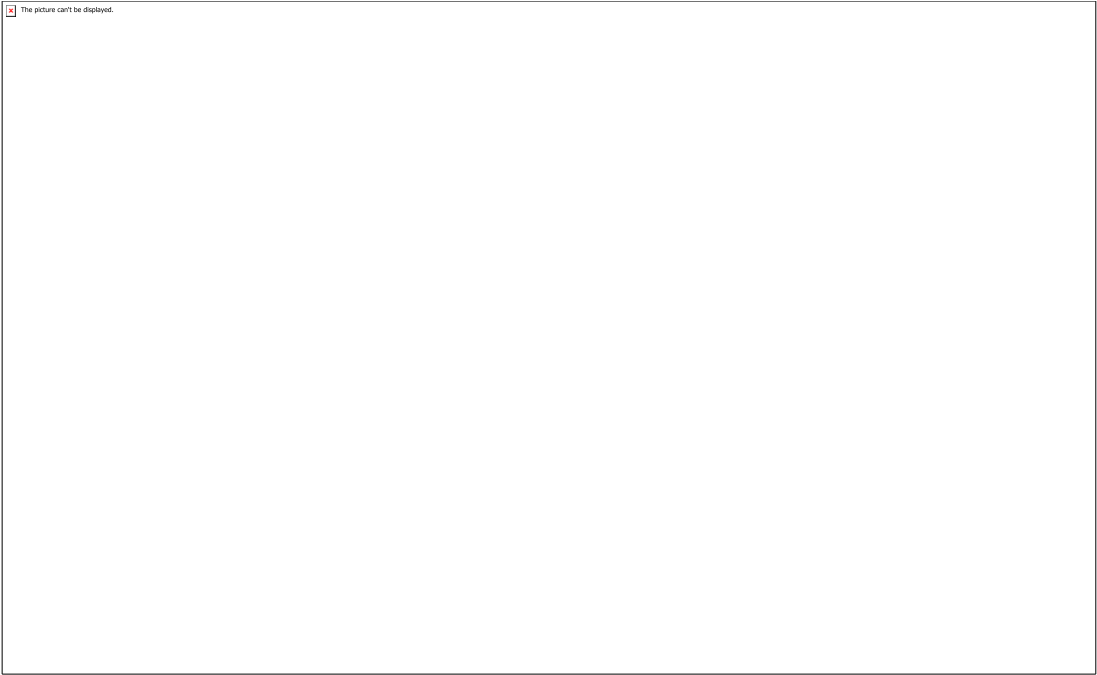


Figure 3.7 : Diagramme de cas d'utilisation global de MOUVE

L'analyse des besoins vise à identifier les services attendus par les utilisateurs et les contraintes que le système doit respecter. MOUVE doit être utilisable par un utilisateur final souhaitant compresser un fichier, par un développeur intégrant le package Python, par un administrateur consultant les journaux de sécurité et par un service externe consommant l'API REST.

La plateforme doit fournir une chaîne complète depuis l'upload jusqu'à la restitution du conteneur. Elle doit aussi préserver l'isolation multi-tenant, gérer les erreurs de manière explicite, produire des métadonnées exploitables et permettre l'extension future vers des profils de compression spécialisés.

3.1.1 Besoins fonctionnels

Les besoins fonctionnels de MOUVE comprennent la compression, la décompression, la classification, le scan de sécurité, la génération de conteneurs, l'authentification, la consultation de l'historique et le traitement batch. Chaque besoin correspond à une capacité observable du système et doit pouvoir être validé par test.

Tableau 3.1 : Besoins fonctionnels de MOUVE

Code	Besoin fonctionnel	Description
F1	Compression	Produire un conteneur .mouve ou .nvc à partir d'un fichier.

F2	Décompression	Restaurer le fichier original sans perte.
F3	Scan sécurité	Analyser le fichier avant traitement intensif.
F4	Interfaces	Exposer API, CLI, Desktop et package Python.
F5	Batch	Traiter plusieurs fichiers de manière asynchrone.

3.1.2 Besoins non fonctionnels

Les besoins non fonctionnels définissent la qualité attendue du système. La performance doit permettre une compression acceptable sur des fichiers de taille moyenne. La sécurité doit réduire les risques associés aux uploads. La maintenabilité doit être assurée par une séparation claire des modules. La scalabilité doit être envisagée par l'usage de workers et de files de tâches.

L'intégrité constitue une exigence centrale. Aucun résultat de décompression ne doit être accepté si le hash restauré ne correspond pas au hash original. La confidentialité est assurée par le format .nvc lorsque le mode sécurisé est activé. Ces propriétés s'appuient sur des standards cryptographiques reconnus [4], [5], [6].

3.1.3 Synthèse des besoins

La synthèse des besoins met en évidence une plateforme à double nature. D'une part, MOUVE est un moteur de compression sans perte. D'autre part, il s'agit d'un système de transfert et de gouvernance de fichiers. Cette double nature explique la nécessité de combiner algorithmes, sécurité applicative, architecture API et mécanismes d'audit.

3.2 Architecture globale du système

3.2.1 Diagramme de classes du domaine

Le diagramme de classes décrit les collaborations structurantes du domaine. ApiService orchestre les opérations mais ne contient pas la logique algorithmique. SecurityScanner agrège les résultats des moteurs de détection et délègue la décision à une politique explicite. Compressor encapsule la sélection de stratégie et s'appuie sur FileClassifier pour définir le profil de traitement. ContainerWriter traduit le résultat vers les formats .mouve ou .nvc, tandis que TenantStorage applique l'isolation des utilisateurs.

Cette répartition suit le principe de responsabilité unique. Elle limite le couplage entre la couche HTTP et les algorithmes, améliore les tests unitaires et permet de remplacer un composant sans réécrire l'ensemble. Une attention particulière doit être accordée aux objets de métadonnées partagés : leur schéma doit être versionné pour garantir la compatibilité ascendante des conteneurs.

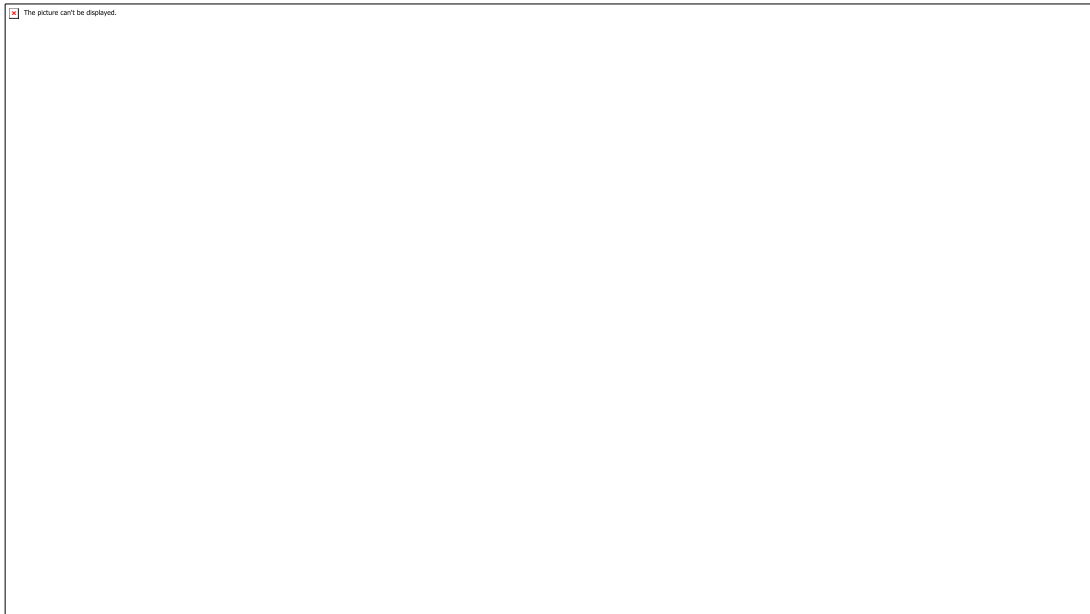


Figure 3.8 : Diagramme de classes simplifié de MOUVE

L'architecture globale repose sur une API centrale qui orchestre les traitements. Les clients peuvent être l'interface Desktop, la CLI, un script Python ou un service externe. L'API applique l'authentification, le rate limiting et le scan de sécurité avant de transmettre le fichier au moteur de compression. Le résultat est stocké avec ses métadonnées dans un espace isolé par utilisateur.

Les traitements longs peuvent être délégués à Celery, qui est conçu comme un système distribué de files de tâches pour traiter de grands volumes de messages et supporter le traitement temps réel ou planifié [12]. Cette architecture permet de maintenir la réactivité de l'API tout en traitant des lots de fichiers.

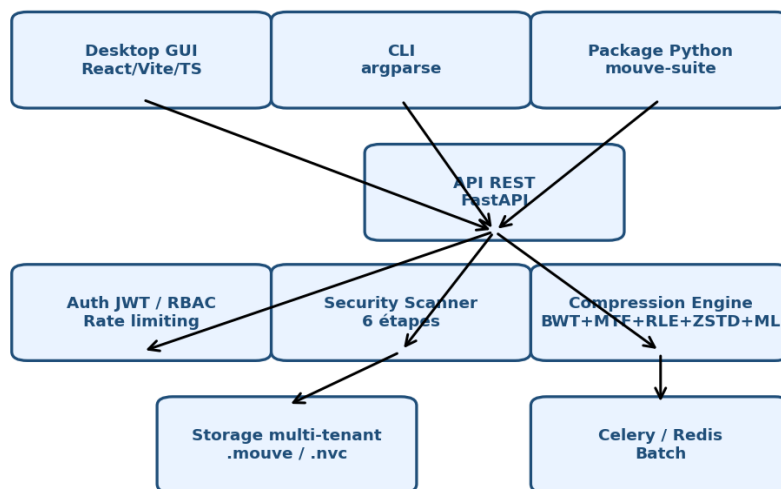


Figure 3.1 : Architecture globale du système MOUVE

3.3 Architectures partielles par étape

3.3.6 Diagramme de séquence

Le scénario nominal de compression commence par une requête multipart envoyée au backend. L'API valide l'identité et les limites de requête, puis transmet le fichier au scanner. Une décision clean autorise l'appel du moteur; une décision suspicious provoque la quarantaine; une décision malicious interrompt le traitement. Le

moteur retourne le payload et les métadonnées au writer, qui construit le conteneur. Le stockage persiste ensuite le fichier dans l'espace du tenant avant que l'API ne renvoie la réponse.

Cette séquence identifie plusieurs points de défaillance : indisponibilité du scanner, timeout du moteur, écriture partielle du conteneur, échec du stockage et interruption réseau. La réalisation doit donc employer des opérations atomiques, des fichiers temporaires, des timeouts et un nettoyage systématique. En batch, l'API renvoie un identifiant de tâche et le worker exécute la même orchestration de manière idempotente.

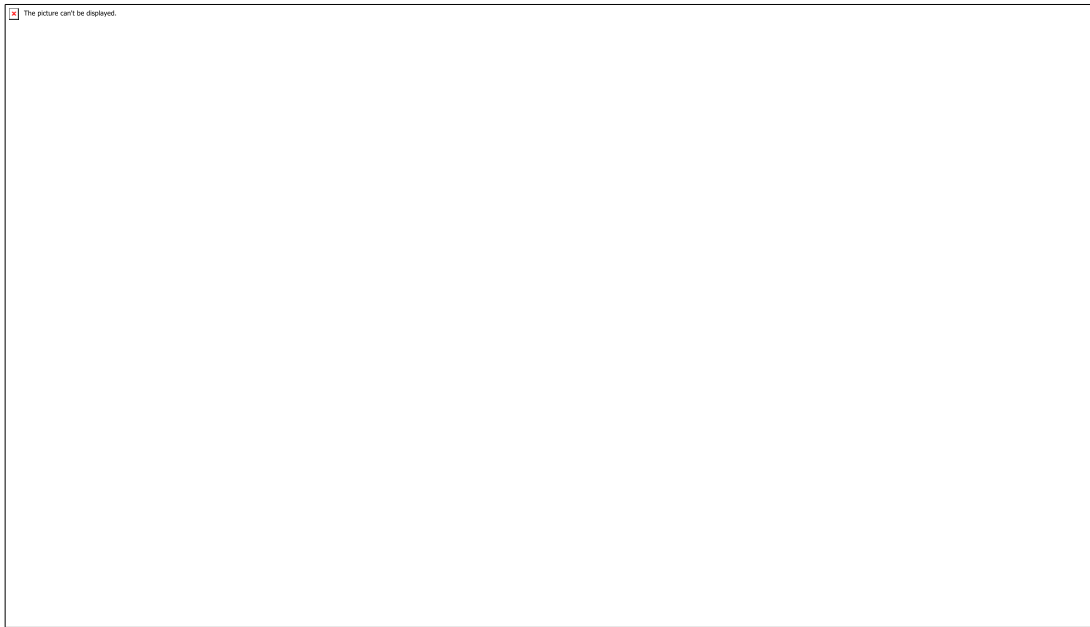


Figure 3.9 : Diagramme de séquence de la compression sécurisée

Les architectures partielles décrivent la décomposition du système en modules. Cette décomposition est nécessaire pour gérer la complexité du projet. Chaque module possède une responsabilité propre, des entrées, des sorties et des points de validation. La conception modulaire facilite également les tests unitaires et les évolutions futures.

3.3.1 Étape 1 : construction et analyse du fichier

La première étape concerne la lecture du fichier, l'extraction d'un échantillon et la classification. Le classificateur analyse les premiers octets, calcule le ratio de caractères imprimables et l'entropie de Shannon. Cette information permet d'orienter le choix de niveau de compression et de détecter les données potentiellement aléatoires.

Cette étape est volontairement légère afin de ne pas retarder inutilement l'utilisateur. Elle fournit cependant des métadonnées précieuses pour la suite, notamment la catégorie estimée du fichier, le niveau de confiance et le profil de compression suggéré.

3.3.2 Étape 2 : moteur de compression classique

Le moteur classique applique la chaîne BWT, MTF, RLE et ZSTD lorsque cette stratégie est pertinente. Il conserve les métadonnées nécessaires à l'inversion, notamment l'index BWT, les flags de pipeline et les informations de backend. En parallèle, une compression Zstandard directe est effectuée afin de déterminer si le pipeline complet apporte un gain réel.

La sélection finale se fait sur la taille du payload et non sur une hypothèse théorique. Cette décision rend le système plus robuste car elle tient compte du fichier réel. Le conteneur enregistre le backend choisi afin que la décompression applique exactement l'inverse de la stratégie retenue.



Stratégie intelligente : comparaison du pipeline complet avec ZSTD direct puis conservation du résultat le plus compact.

Figure 3.2 : Architecture partielle du moteur de compression classique

3.3.3 Étape 3 : mode appris

Le mode appris repose sur un modèle PyTorch capable de produire une représentation latente compacte. PyTorch fournit une bibliothèque optimisée pour les tenseurs et l'apprentissage profond sur CPU et GPU, ce qui le rend adapté à l'expérimentation de modèles d'encodage [10]. Le modèle est entraîné hors ligne sur des corpus représentatifs, puis utilisé seulement lorsque sa performance est démontrée.

La conception impose une validation stricte : toute sortie reconstruite doit être vérifiée par SHA-256. En cas d'échec ou d'absence de modèle compatible, le système revient au pipeline classique. Cette approche garantit que l'innovation apportée par l'IA ne compromet pas la fiabilité du système.



Le mode appris est réservé aux familles de fichiers répétitives et évalué par benchmark avant activation.

Figure 3.3 : Architecture partielle du mode appris

3.3.4 Étape 4a : API REST et traitement batch

L'API REST est développée avec FastAPI, un framework Python moderne reposant sur les annotations de type et générant une documentation interactive compatible OpenAPI [8], [9]. Elle expose les endpoints de compression, de décompression, de batch, de consultation des fichiers, de santé et d'administration.

Le traitement batch est délégué à Celery lorsque plusieurs fichiers doivent être compressés. Redis joue le rôle de broker ou de backend selon la configuration. Cette séparation entre requêtes synchrones et tâches asynchrones évite de bloquer l'API pendant les traitements longs [12].

Tableau 3.2 : Principaux endpoints REST de MOUVE

Méthode	Endpoint	Rôle
POST	/compress	Compression d'un fichier.
POST	/decompress	Restitution de l'original.
POST	/batch/compress	Compression asynchrone.

GET	/files	Liste des fichiers utilisateur.
GET	/security/logs	Consultation administrative des logs.

3.3.5 Étape 4b : format conteneur sécurisé

3.3.7 Diagramme d'activité

Le diagramme d'activité représente les décisions successives depuis la réception du fichier jusqu'à l'écriture du conteneur. Il souligne que la validation de sécurité précède la classification et la compression, ce qui évite de consacrer des ressources importantes à un fichier qui doit être bloqué. La branche de refus conduit explicitement à la quarantaine ou au blocage, accompagnés d'un événement d'audit.

La branche autorisée effectue la classification, puis exécute les stratégies de compression candidates. La comparaison doit inclure la taille des métadonnées et non uniquement celle du payload, faute de quoi de petits fichiers pourraient être considérés comme améliorés alors que le conteneur final est plus volumineux. Ce point constitue une justification importante de la métrique de gain net.

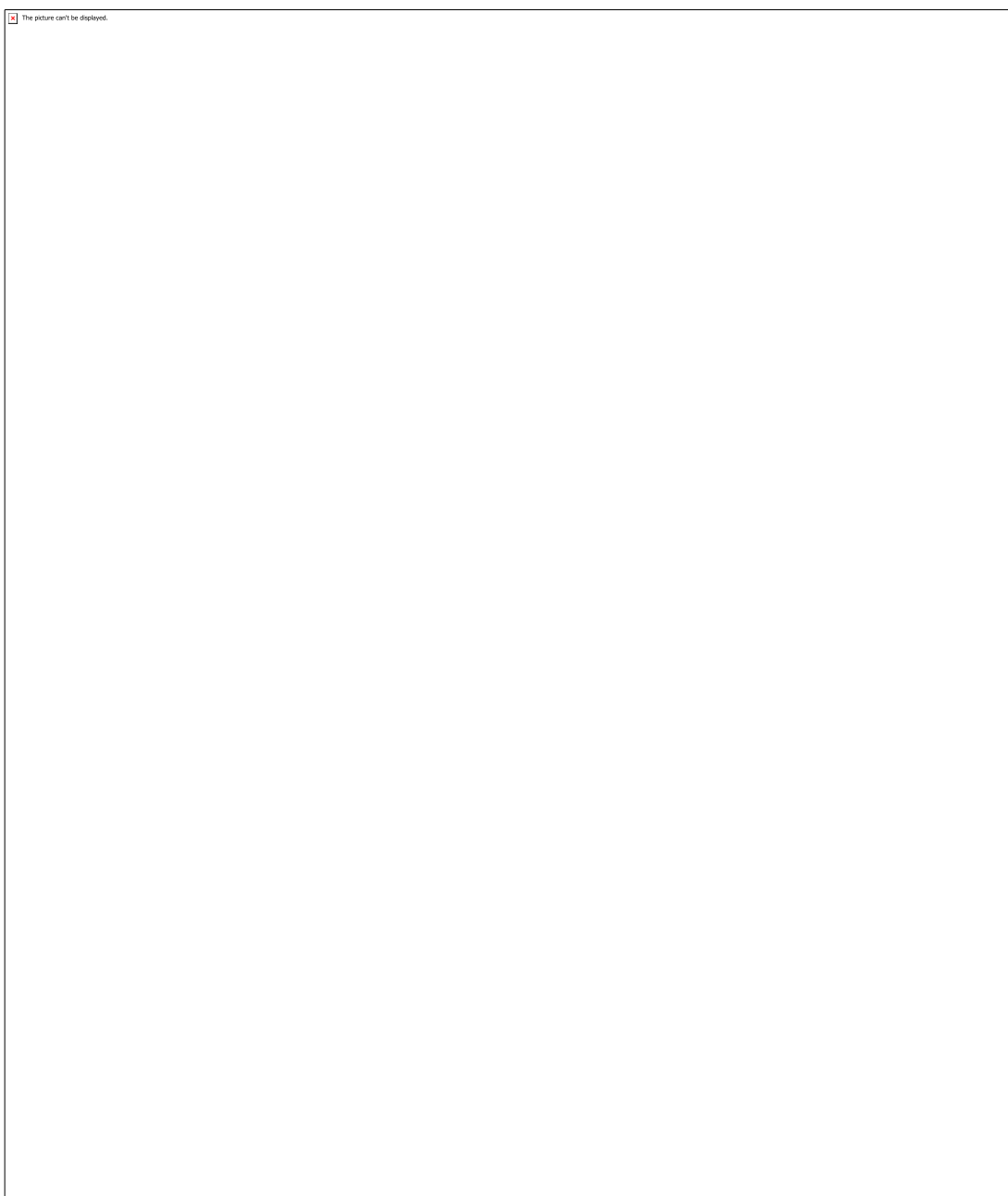


Figure 3.10 : Diagramme d'activité du traitement d'un fichier

Le format conteneur a pour rôle de rassembler le payload compressé et les métadonnées nécessaires à son exploitation. Le format .mouve conserve une structure compacte pour les usages classiques. Le format .nvc ajoute des chunks structurés, un chiffrement authentifié, un hash global et une signature.

Cette distinction permet de concilier simplicité et sécurité. Lorsque l'utilisateur souhaite uniquement compresser localement, le format legacy suffit. Lorsque le fichier doit être transféré ou archivé dans un environnement sensible, le format .nvc apporte des garanties supplémentaires en matière de confidentialité, d'intégrité et de chaîne de custody [4], [5], [6].



Format .nvc : chiffrement AES-256-GCM, signature Ed25519 et hachage global pour l'intégrité.

Figure 3.4 : Format sécurisé .nvc

3.4 Choix technologiques

3.4.4 Diagramme de composants

Le diagramme de composants synthétise les dépendances de déploiement et de développement. Les trois clients — Desktop, CLI et package Python — convergent vers le même domaine de compression, directement ou via l'API. L'API dépend des services d'authentification, du scanner, du moteur et du stockage, tandis que Celery et Redis apportent la capacité de traitement différé. Le format conteneur est considéré comme un composant distinct car il possède ses propres règles de compatibilité, de sécurité et de versionnement.

Cette architecture réduit la duplication fonctionnelle, mais implique la présence de contrats stables. Les types de réponse, les codes d'erreur, les statuts de job et les métadonnées de conteneur doivent être documentés. L'usage d'OpenAPI facilite la description des contrats REST, tandis que le package Python doit exposer une API publique limitée et versionnée [20], [21].

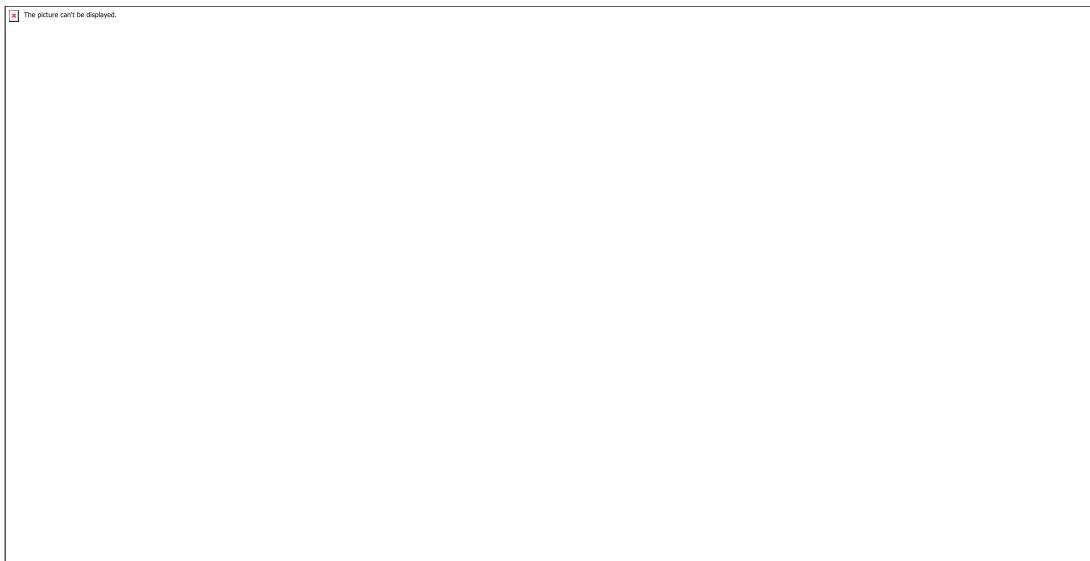


Figure 3.11 : Diagramme de composants de MOUVE

Les choix technologiques de MOUVE sont guidés par la compatibilité Python, la simplicité d'intégration, la maturité des outils et la possibilité de déployer la solution dans un environnement conteneurisé. Le projet privilégie des composants largement utilisés et documentés afin de réduire les risques de maintenance.

3.4.1 Choix matériels

Le projet ne nécessite pas une plateforme matérielle spécialisée. Le développement peut être conduit sur un PC standard disposant de suffisamment de mémoire pour manipuler des fichiers de test. L'entraînement du mode appris peut bénéficier d'un GPU, mais le pipeline classique reste exécutable sur CPU. Cette approche facilite la portabilité du projet et son adoption dans des environnements variés.

3.4.1.1 Serveur de développement

Le serveur de développement héberge l'API FastAPI, les dépendances Python et les scripts de test. Il sert également de point d'entrée pour les benchmarks et les scénarios de sécurité. Dans un environnement industrialisé, ce rôle peut être assuré par un conteneur Docker exécuté sur une machine locale, un serveur d'intégration ou un cluster.

3.4.1.2 Poste client et interface Desktop

Le poste client exécute l'interface Desktop basée sur React, Vite et TypeScript. Vite fournit un serveur de développement rapide et un build optimisé pour les projets web modernes [14]. TypeScript permet de renforcer la maintenabilité du front-end en détectant certaines erreurs avant exécution.

3.4.1.3 Stockage et isolation multi-tenant

Le stockage doit garantir que chaque utilisateur accède uniquement à ses propres fichiers. Cette exigence est traitée par une organisation par tenant et par une validation stricte des identifiants de fichiers. Les chemins directs ne doivent pas être exposés aux utilisateurs, afin de limiter les risques de traversal ou d'accès non autorisé [7].

3.4.2 Choix logiciels

Les logiciels retenus couvrent les couches backend, frontend, IA, sécurité et déploiement. Python est choisi pour le moteur et l'API. React, Vite et TypeScript sont utilisés pour l'interface Desktop. Docker facilite la reproductibilité, tandis que Celery et Redis permettent les traitements asynchrones.

3.4.2.1 Systèmes d'exploitation

Le développement est compatible avec Linux et Windows, mais l'environnement de référence pour le déploiement reste Linux en raison de sa compatibilité avec Docker, les outils Python et les pipelines CI/CD. Les chemins, permissions et variables d'environnement doivent être abstraits afin de conserver la portabilité.

3.4.2.2 Python

Python constitue le langage principal du moteur de compression, de la CLI, de l'API et des expérimentations IA. Son écosystème facilite l'intégration de Zstandard, de bibliothèques cryptographiques, de frameworks web et de PyTorch. Cette homogénéité réduit le coût d'intégration entre les modules.

3.4.2.3 FastAPI

FastAPI est retenu car il permet de créer rapidement des API typées et documentées. Il repose sur des standards ouverts comme OpenAPI et JSON Schema, ce qui facilite la génération de documentation, les tests et l'intégration avec des clients externes [8], [9].

3.4.2.4 React, Vite et TypeScript

L'interface Desktop utilise React pour organiser l'interface en composants, Vite pour accélérer le développement et TypeScript pour améliorer la fiabilité du code. Cette pile technologique est adaptée à une interface de glisser-déposer, d'affichage des résultats et de suivi d'historique [14].

3.4.2.5 Docker, Celery et Redis

3.4.5 Diagramme de déploiement

Le déploiement cible sépare le point d'entrée TLS, les instances FastAPI, les workers Celery, Redis, le stockage persistant et le système de journalisation. Cette séparation permet de dimensionner indépendamment l'API et les traitements de compression. Elle facilite également l'application du moindre privilège : les workers accèdent aux files et au stockage, tandis que le reverse proxy ne manipule jamais directement les données persistées.

Dans Kubernetes, les secrets cryptographiques ne doivent pas être intégrés aux images. Les volumes persistants, probes de santé, limites CPU/mémoire et politiques réseau doivent être définis explicitement. Redis peut être déployé avec haute disponibilité ou remplacé par un service managé selon le niveau de criticité [25], [28], [29].

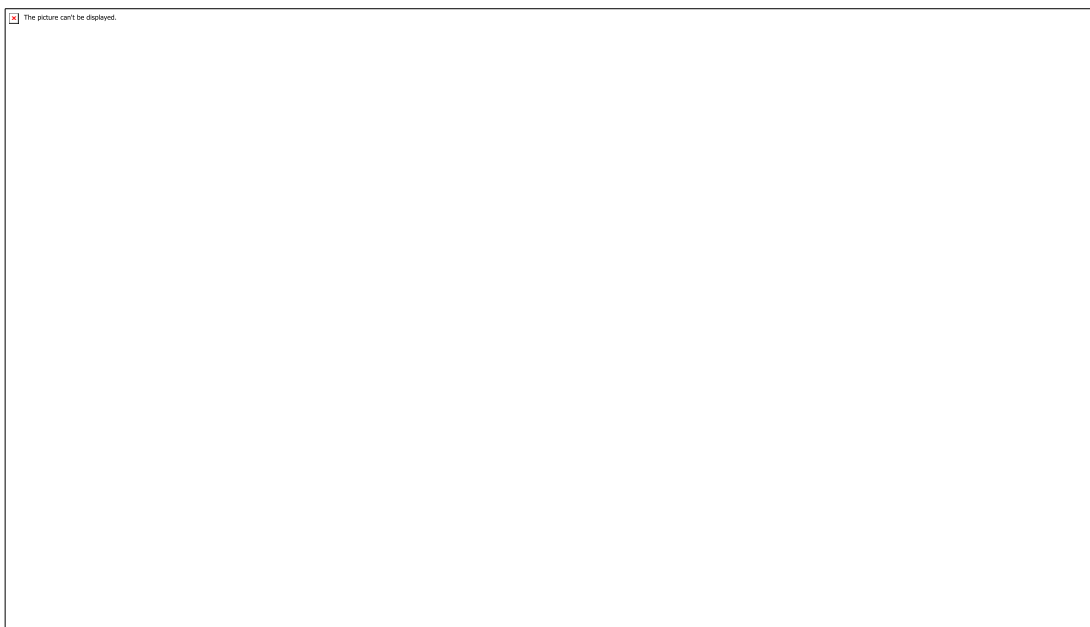


Figure 3.12 : Diagramme de déploiement cible de MOUVE

Docker est choisi pour standardiser l'environnement d'exécution et limiter les écarts entre développement et déploiement. La documentation officielle présente Docker comme une plateforme permettant de conteneuriser les applications et d'optimiser les workflows de développement [13]. Celery et Redis complètent cette architecture pour les tâches asynchrones [12].

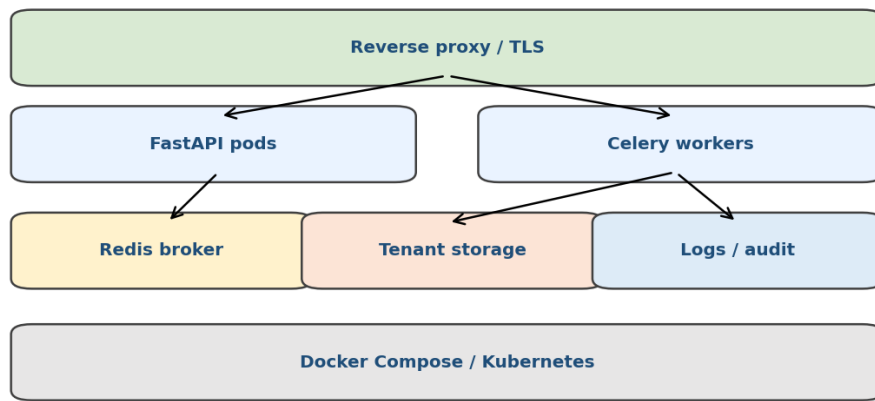


Figure 3.5 : Architecture de déploiement cible

3.4.2.6 PyTorch

PyTorch est utilisé dans le mode appris pour entraîner et exécuter les modèles d'encodage. Sa prise en charge des tenseurs, de l'autograd et des accélérateurs le rend adapté à l'expérimentation de modèles de représentation latente [10].

3.4.2.7 Sécurité logicielle

Les composants de sécurité comprennent le contrôle des fichiers entrants, l'analyse des archives, le hachage SHA-256, le chiffrement AES-GCM, les signatures Ed25519 et la journalisation. Les recommandations OWASP relatives aux uploads justifient la validation de la taille, du type et du contenu avant traitement [7]. Les standards NIST et RFC justifient les choix cryptographiques [4], [5], [6].



Décision : clean si risque < 0,55 ; suspicious entre 0,55 et 0,85 ; malicious à partir de 0,85.

Figure 3.6 : Architecture de sécurité multicouche

3.4.3 Bibliothèques Python utilisées

Les bibliothèques Python utilisées peuvent être regroupées en quatre familles. La première concerne l'API et l'orchestration avec FastAPI, Uvicorn et Pydantic. La deuxième concerne la compression avec Zstandard et les modules internes BWT, MTF et RLE. La troisième concerne la sécurité avec les bibliothèques de hachage et de cryptographie. La quatrième concerne l'IA avec PyTorch et les scripts d'entraînement.

Cette organisation reflète la conception modulaire du projet. Chaque dépendance doit être versionnée et documentée afin de garantir la reproductibilité. Les dépendances critiques doivent faire l'objet d'une veille de sécurité avant un déploiement en production.

Tableau 3.3 : Bibliothèques Python principales

Famille	Bibliothèques	Rôle
API	FastAPI, Uvicorn, Pydantic	Exposition des services REST.
Compression	zstandard, modules internes	Compression et décompression.
Sécurité	hashlib, cryptography	Hash, chiffrement et signature.
IA	torch, numpy	Mode appris et expérimentation.
Asynchrone	celery, redis	Batch et files de tâches.

3.2.2 Flux de données et frontières de confiance

Le flux de données traverse plusieurs frontières de confiance. Le navigateur ou la CLI est non fiable; le reverse proxy limite la taille et termine TLS; l'API valide le token et le tenant; le scanner traite le contenu comme hostile; le moteur ne reçoit qu'un fichier autorisé; le stockage persiste un identifiant interne plutôt qu'un nom fourni par l'utilisateur.

Chaque transition doit être associée à une validation et à une journalisation adaptées. Les métadonnées issues du client sont considérées comme indicatives. Le type réel est détecté côté serveur. Les chemins sont générés par l'application. Les décisions sécurité sont immuables dans le journal d'audit.

Cette analyse de flux réduit les risques de confusion de privilèges, de path traversal et de traitement d'un format inattendu. Elle facilite également la modélisation des menaces.

3.2.3 Modèle de menace STRIDE appliqué à MOUVE

La méthode STRIDE permet de structurer l'analyse autour de l'usurpation, l'altération, la répudiation, la divulgation, le déni de service et l'élévation de privilèges. L'usurpation est traitée par JWT et validation des signatures. L'altération est détectée par AES-GCM, SHA-256 et Ed25519. La répudiation est réduite par des journaux horodatés et chaînés.

La divulgation est limitée par le chiffrement, l'isolation tenant et les contrôles d'autorisation objet. Le déni de service est atténué par les limites de taille, quotas, timeouts, workers et contrôle des archives. L'élévation de privilèges est traitée par RBAC, séparation des rôles et moindre privilège des conteneurs.

Le modèle de menace doit être révisé lors de l'ajout d'un nouveau parser, d'un stockage cloud ou d'un modèle IA externe, car chaque dépendance modifie la surface d'attaque.

3.3.8 Gestion des erreurs et résilience

Les erreurs sont classées en erreurs utilisateur, erreurs de sécurité, erreurs de format, indisponibilités d'infrastructure et défauts internes. Cette taxonomie permet de retourner des codes adaptés, d'éviter les retries inutiles et de déclencher des alertes lorsque l'infrastructure est dégradée.

Les tâches Celery doivent être idempotentes. Un retry ne doit pas créer plusieurs conteneurs ou écraser une version valide. Le job utilise un identifiant stable, un état transactionnel et des fichiers temporaires. Les erreurs définitives conduisent à un état failed documenté, tandis que les erreurs temporaires sont retentées avec backoff.

La résilience inclut le nettoyage des fichiers temporaires, la limitation des files, la gestion des tâches bloquées et la capacité à restaurer les métadonnées à partir du stockage persistant.

3.4.6 Choix architecturaux et compromis

L'architecture modulaire améliore la maintenabilité mais multiplie les contrats et les points d'intégration. Le recours à Celery et Redis apporte de la scalabilité mais augmente la complexité opérationnelle. Le format .nvc améliore la sécurité mais crée une dépendance à la gestion des clés et à la compatibilité du parser.

Le choix d'un monolithe modulaire pour le cœur est adapté au stade PFE, car il réduit les coûts réseau et de déploiement tout en préservant les frontières internes. Une évolution vers des microservices ne serait justifiée que par des besoins de montée en charge, d'équipes indépendantes ou de cycles de déploiement distincts.

La justification architecturale repose donc sur la proportionnalité : introduire uniquement les mécanismes dont la valeur dépasse leur coût de complexité.

Chapitre 3. Conclusion

Ce chapitre a détaillé la conception de MOUVE et les choix technologiques retenus. L'architecture proposée répond aux exigences de modularité, de sécurité et d'intégration multi-interface. Les décisions prises préparent la phase de réalisation, présentée dans le chapitre suivant, où les composants sont assemblés pour former une plateforme exploitable.

Chapitre 4

Réalisation

Plan du chapitre :

- 1 Choix techniques ;
- 2 Plateformes pour l'exécution ;
- 3 Travail réalisé ;
- 4 Planning réel du projet.

Le chapitre présente la concrétisation de la conception, les composants développés et l'évaluation globale de la solution.

Chapitre 4. Réalisation

Introduction

Ce chapitre présente la réalisation de MOUVE. Il décrit les choix techniques appliqués lors du développement, les plateformes d'exécution, les modules effectivement réalisés et le planning réel du projet. L'objectif est de montrer que la solution conçue au chapitre précédent a été traduite en composants cohérents, testables et intégrables.

4.1 Choix techniques

4.1.1 Architecture détaillée du backend FastAPI

Le backend est organisé selon une architecture en couches. Les routers traduisent les requêtes HTTP en commandes applicatives et ne contiennent pas la logique de compression. Les dépendances FastAPI centralisent l'authentification JWT, l'autorisation RBAC, la limitation de débit et le contexte tenant. Les services orchestrent les workflows de compression, de sécurité et de fichiers. Le domaine regroupe les transformations, la classification, le format et la décision de risque. L'infrastructure encapsule le stockage, Redis, Celery et les journaux.

Cette organisation rend les tests plus précis. Les routers peuvent être testés avec un client HTTP et des services simulés, tandis que les algorithmes sont testés sur des vecteurs déterministes. Les transactions de fichiers doivent utiliser une écriture temporaire suivie d'un renommage atomique. Les exceptions internes sont converties en réponses HTTP cohérentes sans exposer de chemins, de clés ou de traces sensibles.

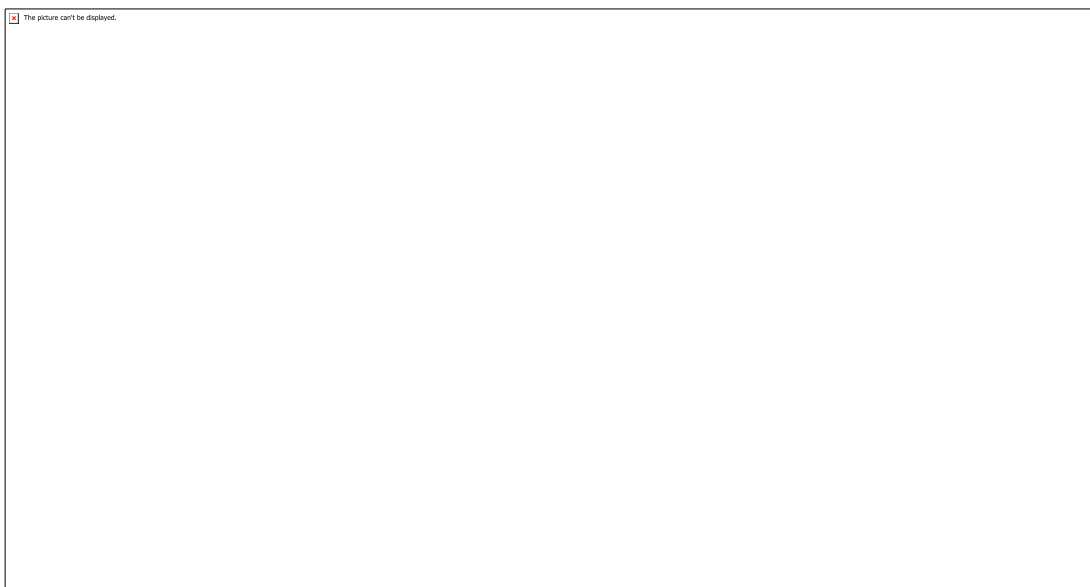


Figure 4.4 : Architecture interne du backend FastAPI

Les choix techniques de réalisation se fondent sur la séparation entre moteur, API, sécurité, interface et déploiement. Le moteur de compression est isolé dans un module dédié afin d'être utilisé à la fois par l'API, la CLI

et le package Python. Cette décision évite la duplication et garantit que toutes les interfaces produisent des résultats cohérents.

Le scanner de sécurité est positionné avant le moteur afin de réduire les risques liés aux fichiers entrants. Les formats conteneurs sont implémentés séparément afin de permettre l'évolution du format .nvc sans casser le format .mouve. L'API sert d'orchestrateur et délègue les opérations longues à Celery lorsque nécessaire.

4.2 Plateformes pour l'exécution de MOUVE

MOUVE peut être exécuté selon plusieurs modes. En développement local, l'API fonctionne avec Uvicorn et l'interface Desktop avec Vite. En mode automatisé, la CLI permet de lancer des compressions depuis un terminal ou un script. En mode production, Docker Compose peut orchestrer l'API, Redis, les workers et le stockage. Cette flexibilité répond aux besoins des utilisateurs et des développeurs.

4.2.1 Critères de sélection

Les plateformes d'exécution sont comparées selon la latence, la facilité de déploiement, la compatibilité avec les dépendances Python, la possibilité d'exécuter des tâches asynchrones et la sécurité opérationnelle. Un environnement de production doit également permettre la gestion des secrets, la supervision, les sauvegardes et les journaux d'audit.

Tableau 4.1 : Critères de sélection des plateformes

Critère	Importance pour MOUVE
Latence	Détermine la réactivité lors des compressions interactives.
Scalabilité	Permet d'absorber les opérations batch.
Portabilité	Facilite le déploiement sur différents environnements.
Sécurité	Protège fichiers, clés et métadonnées.
Observabilité	Permet de suivre erreurs, ratios et décisions sécurité.

4.2.2 Serveur API FastAPI

Le serveur API représente la plateforme principale d'exécution. Il reçoit les requêtes, valide l'utilisateur, applique la politique de sécurité, déclenche le moteur de compression et retourne le résultat. FastAPI facilite cette implémentation grâce aux modèles typés et à la documentation interactive [8], [9].

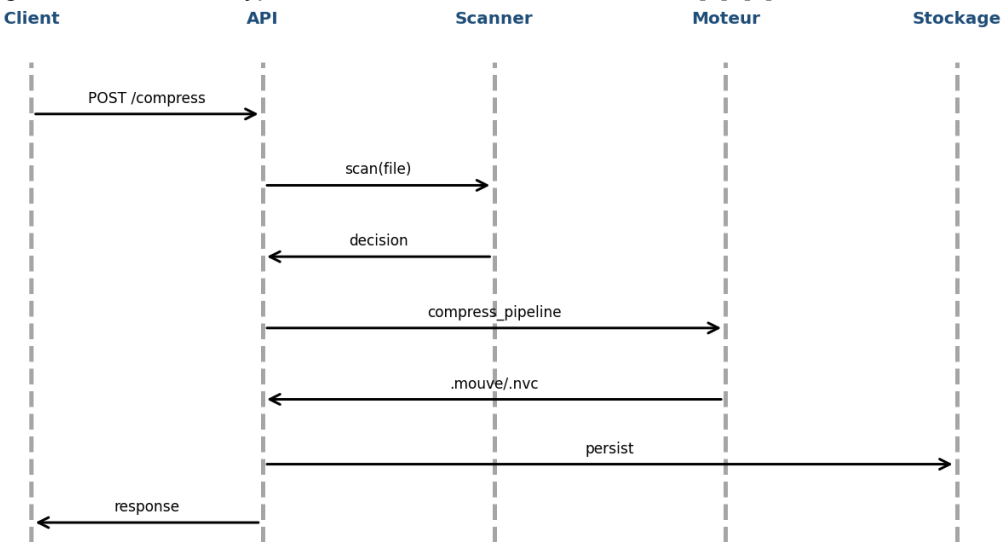


Figure 4.1 : Diagramme de séquence de la compression via API

4.2.3 Interface Desktop

4.2.3.1 Architecture frontend React et Desktop

Le frontend est structuré par fonctionnalités plutôt que par catégories de fichiers. Les modules compression, décompression, historique et authentification possèdent leurs composants, hooks, types et appels API. Le client HTTP centralise la base URL, l'injection du token, la gestion des erreurs et l'annulation des requêtes. Les états transitoires — upload, progression, succès et erreur — sont explicitement représentés afin d'éviter les interfaces ambiguës.

L'application Desktop réutilise le frontend React et l'encapsule dans l'environnement choisi par le projet. Le drag-and-drop doit effectuer une validation initiale sans être considéré comme une barrière de sécurité; le backend reste l'autorité. Vite fournit le serveur de développement et le build, tandis que TypeScript renforce les contrats entre composants et réponses API [22], [23].

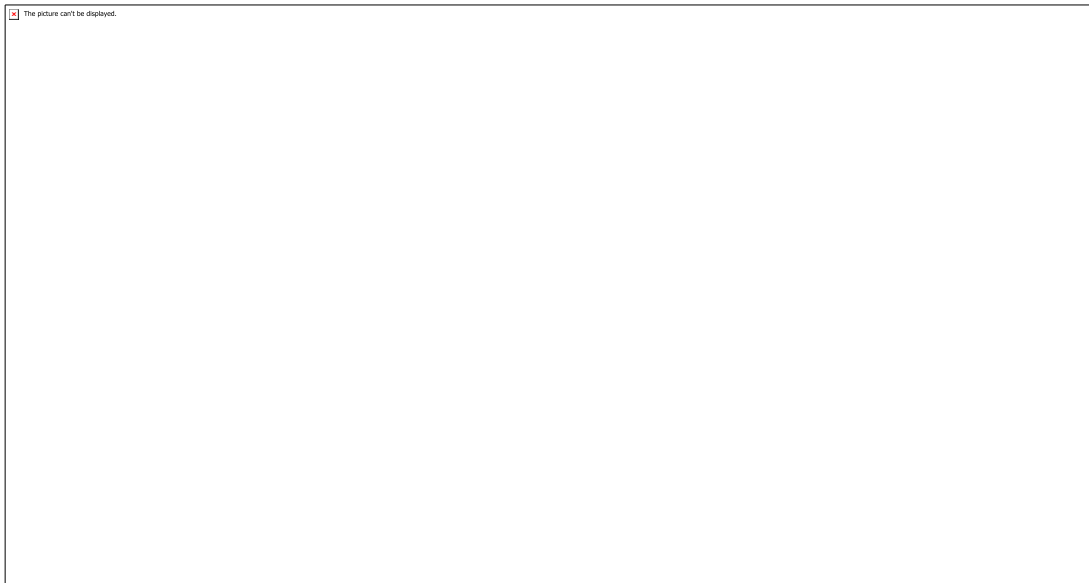


Figure 4.5 : Architecture frontend React/Vite/TypeScript

L'interface Desktop apporte une expérience utilisateur simplifiée. Elle masque la complexité du pipeline et présente à l'utilisateur les informations essentielles : fichier choisi, action demandée, statut, taille originale, taille compressée et gain obtenu. Le choix React/Vite/TypeScript permet de conserver une interface maintenable et évolutive [14].

4.2.4 PC de développement et package Python

4.2.4.1 Architecture de la CLI et du package Python

La CLI constitue une façade mince au-dessus du package Python. Elle valide les arguments, ouvre les fichiers en mode binaire, appelle l'API publique du moteur et traduit les exceptions en codes de sortie exploitables par les scripts. Les commandes compress, decompress et train-ml doivent produire des messages déterministes, accepter des chemins explicites et interdire l'écrasement silencieux d'un fichier existant.

Le package mouve-suite expose un nombre réduit de fonctions stables. Les détails internes de BWT, MTF, RLE et du conteneur restent encapsulés. Cette séparation réduit le risque que les applications clientes dépendent de structures internes susceptibles d'évoluer. La documentation doit préciser les types, exceptions, garanties d'intégrité et versions de format compatibles.

Le PC de développement constitue l'environnement de référence pour écrire, tester et documenter les modules. Le package Python permet d'utiliser le moteur dans d'autres applications, ce qui renforce la réutilisabilité du projet. Cette forme d'intégration est importante pour les scénarios de recherche, de benchmark et d'automatisation.

```
from mouve_suite.mouve_core import compressor
```

```
compressed, meta = compressor.compress_pipeline(data)
restored = compressor.decompress_pipeline(compressed, meta)
```

4.2.5 Comparaison des plateformes cibles

La comparaison des modes d'exécution montre que l'API est adaptée à l'intégration applicative, la CLI à l'automatisation, le Desktop à l'usage interactif et le package Python aux développeurs. Le déploiement Docker offre une couche de portabilité permettant de rapprocher l'environnement de développement et l'environnement de production [13].

Tableau 4.2 : Comparaison des plateformes d'exécution

Mode	Avantage principal	Limite principale
API REST	Intégration simple avec systèmes externes	Nécessite un serveur actif
CLI	Automatisation rapide	Moins adaptée aux utilisateurs non techniques
Desktop	Expérience utilisateur directe	Dépend de l'API backend
Package Python	Réutilisation dans du code	Nécessite des compétences Python
Docker	Portabilité et reproductibilité	Demande une configuration initiale

4.3 Travail réalisé

4.3.3 Endpoints REST et comportement détaillé

L'endpoint POST /compress reçoit un fichier multipart, applique l'authentification, le rate limiting et le scan, puis retourne directement le conteneur ou un identifiant selon le mode. POST /decompress valide le magic, la version, les tailles et les hashes avant reconstruction. POST /batch/compress crée un job idempotent et GET /batch/{job_id} expose un état fini ou transitoire sans divulguer les détails d'un autre tenant.

Les endpoints /files et /files/{file_id} appliquent systématiquement l'autorisation objet. /health distingue idéalement liveness et readiness afin de ne pas déclarer prêt un service incapable de joindre Redis ou le stockage. Les endpoints administratifs /security/logs, /security/stats et /admin/users nécessitent un rôle admin et doivent être audités [20], [21].

Tableau 4.3 : Contrats et contrôles des endpoints REST

Méthode	Endpoint	Validation critique	Réponse attendue
POST	/compress	auth, taille, MIME, scan	conteneur ou erreur 4xx
POST	/decompress	magic, version, hash	fichier original
POST	/batch/compress	auth, quotas, idempotence	job_id
GET	/batch/{job_id}	tenant, existence	état du job
GET	/health	dépendances critiques	statut opérationnel
GET	/security/logs	rôle admin	événements filtrés

4.3.2 Mécanismes de sécurité réalisés

Le scanner applique une défense en profondeur. L'ingress gate impose les limites de taille et la politique MIME; l'analyse d'archive contrôle le nombre d'entrées, la profondeur, le ratio d'expansion et les chemins; la réputation compare SHA-256 aux listes; ClamAV et YARA recherchent des signatures et motifs; les heuristiques examinent les exécutables, scripts et polyglots; le scoring agrège les signaux. Cette organisation correspond aux recommandations OWASP et aux capacités documentées de ClamAV et YARA [11]–[14].

Le score ne doit pas être interprété comme une probabilité statistique tant qu'il n'est pas calibré sur un corpus étiqueté. Il constitue initialement une mesure de risque pondérée. La documentation doit distinguer les décisions déterministes — par exemple un hash interdit — des décisions heuristiques. Cette distinction améliore l'explicabilité et facilite l'analyse des faux positifs.

4.3.1 Moteur de compression et stratégie de sélection

La réalisation du moteur doit distinguer les blocs, les transformations et la politique de sélection. Chaque transformation est testée par aller-retour afin de vérifier son inversion. Le benchmark interne mesure la taille du résultat direct et celle du pipeline complet, incluant l'en-tête. Le choix final est enregistré dans les métadonnées. Cette conception évite l'erreur méthodologique consistant à annoncer un gain sur la seule sortie intermédiaire.

La complexité de BWT impose un traitement par blocs et, pour les volumes importants, une implémentation de suffix array optimisée. Une version naïve fondée sur toutes les rotations n'est acceptable que pour des tests pédagogiques. La discussion des performances doit donc séparer la validité fonctionnelle de l'industrialisation algorithmique [1]–[3].

Le travail réalisé comprend la définition du moteur de compression, l'intégration de Zstandard, la création des structures de métadonnées, la conception du format conteneur, l'implémentation de l'API, la préparation de la CLI, la conception de l'interface Desktop et l'intégration du scanner de sécurité. Chaque composant a été conçu pour être testable séparément.

Les tests ont porté sur la décompression sans perte, la cohérence des métadonnées, la sélection de stratégie, la gestion des fichiers incompressibles, la détection des fichiers suspects et la restitution des résultats à travers les interfaces. Cette phase a également permis de produire des graphes de ratios et de préparer la documentation finale.

Ratio de compression attendu par famille de fichiers

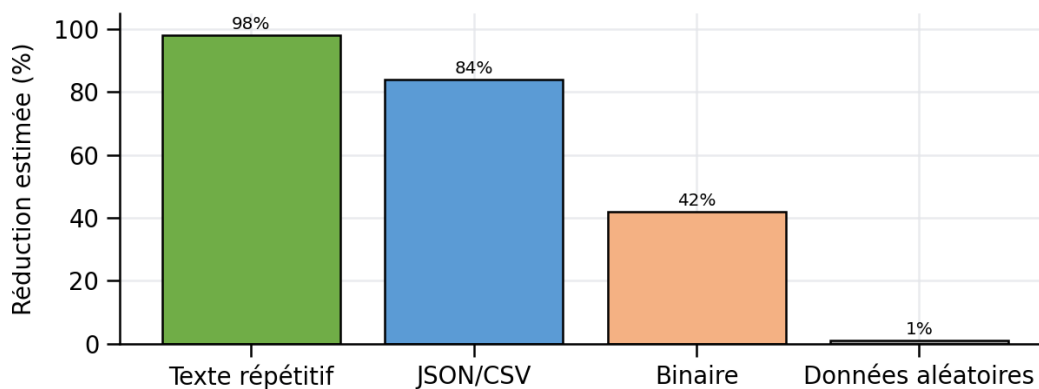


Figure 4.2 : Synthèse graphique des résultats attendus

4.4 Planning réel du projet

4.6 Validation de la partie Intelligence Artificielle

Le dataset du classificateur doit contenir des échantillons TEXT, SOURCE_CODE, BINARY et RANDOM, équilibrés par classe et séparés au niveau des fichiers sources pour éviter la fuite entre entraînement et test. Le prétraitement extrait des fenêtres d'octets, normalise les histogrammes, calcule l'entropie, le ratio imprimable, les fréquences de contrôle et des statistiques de runs. Les décisions de split et la provenance des données doivent être documentées.

Le modèle doit être décrit avec sa dimension d'entrée, ses couches, fonctions d'activation, régularisation et sortie softmax. L'entraînement documente l'optimiseur, le learning rate, la taille de batch, le nombre d'époques, l'early stopping, la graine aléatoire et le matériel. Les métriques minimales sont accuracy, precision, recall, F1 macro, matrice de confusion, loss d'entraînement et de validation. Pour le VQ-VAE, les métriques supplémentaires sont le taux, la taille du codebook, la reconstruction exacte et le coût d'encodage [26], [27].



Figure 4.7 : Gabarits des figures IA à remplacer par les sorties réelles

4.5.2 Tableau de résultats reproductibles à compléter

Le tableau suivant est volontairement laissé sans résultats numériques. Il doit être alimenté par le script de benchmark exécuté sur la configuration décrite dans le rapport. Cette pratique protège la rigueur scientifique du document et permet au jury de distinguer les résultats mesurés des performances attendues ou issues de la littérature.

Tableau 4.5 : Gabarit des résultats de benchmark

Corpus	Algorithme	Ratio	Compression MB/s	Décompression MB/s	Mémoire pic
Texte	ZIP / GZIP / BZIP2 / ZSTD / MOUVE	À mesurer	À mesurer	À mesurer	À mesurer
JSON/CSV	ZIP / GZIP / BZIP2 / ZSTD / MOUVE	À mesurer	À mesurer	À mesurer	À mesurer
Binaire	ZIP / GZIP / BZIP2 / ZSTD / MOUVE	À mesurer	À mesurer	À mesurer	À mesurer
Aléatoire	ZIP / GZIP / BZIP2 / ZSTD / MOUVE	À mesurer	À mesurer	À mesurer	À mesurer

4.5.1 Protocole de benchmark comparatif

Le benchmark doit comparer ZIP/DEFLATE, GZIP, BZIP2, ZSTD direct et MOUVE sur le même matériel, avec les mêmes fichiers, après échauffement et sur plusieurs répétitions. Les mesures minimales sont la taille finale, le ratio, le temps de compression, le temps de décompression, le débit, le pic mémoire et le taux de sélection de chaque stratégie. Les résultats sont rapportés par type de données et par taille afin d'éviter qu'une moyenne globale masque les cas défavorables.

Aucune valeur expérimentale ne doit être inventée. En l'absence des sorties d'exécution et du corpus réel, le rapport présente le protocole, les tableaux à remplir et les critères d'interprétation. Une affirmation de supériorité n'est recevable que si MOUVE améliore significativement le ratio sans dégrader de manière disproportionnée le temps, la mémoire ou la sécurité. Le site Zstandard illustre précisément ce compromis entre ratio et vitesse sur le corpus Silesia [4].

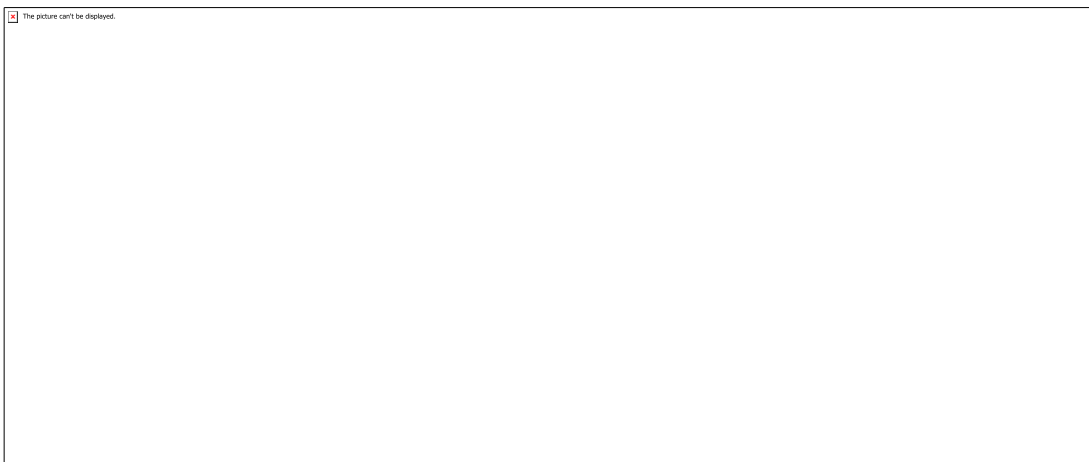


Figure 4.6 : Dimensions du benchmark comparatif à alimenter par mesures réelles

4.5 Validation fonctionnelle, sécurité et performance

La validation est organisée selon le modèle de qualité ISO/IEC 25010 afin de couvrir l'adéquation fonctionnelle, l'efficacité, la fiabilité, la sécurité et la maintenabilité [31], [32]. Les tests fonctionnels vérifient la compression et la décompression de chaque famille de fichiers, les quatre interfaces, les erreurs de format, l'isolation tenant et les jobs batch. L'oracle principal est l'égalité SHA-256 entre l'entrée et la sortie restaurée.

Les tests de sécurité couvrent les extensions trompeuses, MIME falsifiés, fichiers surdimensionnés, archives à expansion élevée, chemins ../, symlinks, archives imbriquées, signatures YARA, échantillons antivirus de test, accès croisé entre tenants, JWT expirés et tentatives d'accès aux routes admin. Chaque cas documente l'entrée, la décision attendue, le code HTTP, l'événement d'audit et le résultat observé [11]–[14].

Tableau 4.4 : Plan de validation fonctionnelle et sécurité

Catégorie	Exemple de scénario	Critère de réussite
Fonctionnel	compress → decompress sur texte, JSON, binaire	hash identique
Sécurité	archive traversal ou zip bomb contrôlée	blocage/quarantaine
API	token expiré ou rôle insuffisant	401/403
Multi-tenant	accès au fichier d'un autre utilisateur	refus sans fuite
Résilience	Redis indisponible pendant un batch	erreur contrôlée/retry

Le planning réel a suivi une progression itérative. Les premières phases ont été consacrées à la compréhension du sujet et à l'état de l'art. L'implémentation du moteur classique a constitué l'étape centrale. La sécurité et les formats conteneurs ont ensuite été ajoutés. Enfin, les interfaces et la rédaction du rapport ont permis de finaliser le projet dans une forme exploitable.

La rédaction académique a accompagné la réalisation afin de conserver une traçabilité des choix. Cette méthode a facilité l'alignement entre les objectifs annoncés, l'architecture conçue, les composants réalisés et les perspectives d'amélioration.

Planning prévisionnel du projet MOUVE

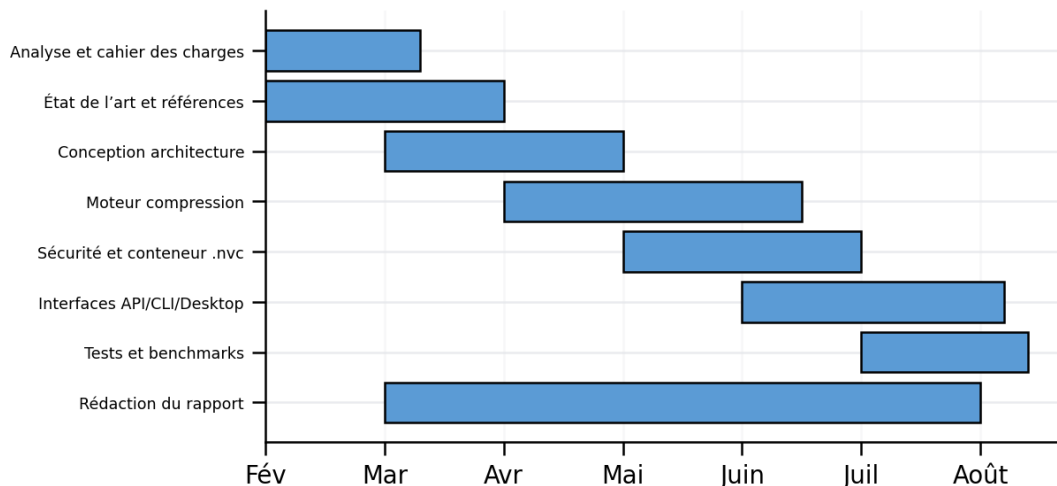


Figure 4.3 : Planning réel et phases de réalisation

4.1.2 Structure des dossiers du backend

La structure du backend peut être organisée autour des répertoires api, core, services, domain, infrastructure, security et tests. Le répertoire api contient les routers et schémas. core centralise la configuration et les dépendances transverses. services orchestre les cas d'utilisation. domain contient les transformations, profils et objets de métadonnées. infrastructure implémente stockage, Redis, Celery et persistance.

Cette organisation doit être accompagnée de règles de dépendance. La couche domaine ne dépend pas de FastAPI ou de Redis. Les services dépendent du domaine à travers des interfaces. L'infrastructure implémente ces interfaces. Cette inversion facilite les tests en mémoire et réduit le couplage technologique.

Les tests reproduisent la même organisation afin que chaque module possède des tests unitaires, d'intégration et de contrat.

4.1.3 Implémentation du moteur BWT–MTF–RLE–ZSTD

L'implémentation par blocs évite de charger ou de trier des structures disproportionnées. Chaque bloc produit un index BWT, un flux MTF, une séquence RLE et un payload ZSTD. Les métadonnées de tous les blocs sont sérialisées dans un ordre déterministe. La décompression vérifie les tailles avant l'allocation afin de limiter les risques de conteneurs malveillants.

La sélection automatique évalue le payload direct et le payload transformé. Elle doit inclure les en-têtes et mesurer le gain net. Un seuil minimal peut éviter de choisir une stratégie plus complexe pour un gain négligeable. Le niveau ZSTD est défini par profil et non codé en dur.

L'implémentation doit assurer la compatibilité avec les fichiers vides, les blocs incomplets, les octets nuls et les fichiers très petits. Ces cas limites sont essentiels pour la fiabilité d'un compresseur.

4.1.4 Implémentation des formats .mouve et .nvc

Le format .mouve adopte un en-tête contenant le magic, la version, les flags, la taille originale, le hash, l'identifiant de pipeline et les métadonnées de blocs. Les entiers utilisent un endianness défini. Toute extension future doit être introduite par version ou chunk optionnel afin d'éviter une interprétation ambiguë.

Le format .nvc est construit comme une suite de chunks typés. META contient les métadonnées authentifiées, DATA le payload chiffré, GSH le hash global, SIGN la signature et END! la terminaison. Chaque chunk porte une longueur contrôlée et un CRC pour la détection précoce de corruption.

Le parser est défensif : il rejette les longueurs incohérentes, chunks dupliqués interdits, versions inconnues et données après END!. Cette discipline réduit la surface d'attaque du décodeur.

4.1.5 Implémentation cryptographique AES-GCM, Ed25519 et SHA-256

AES-GCM nécessite une clé robuste et un nonce unique. Le nonce est généré par un générateur cryptographique et stocké avec le conteneur. Les métadonnées critiques sont placées en AAD afin qu'une modification soit détectée même si elles ne sont pas chiffrées. Le tag est vérifié avant toute décompression.

Ed25519 signe une représentation canonique des métadonnées et du hash du ciphertext. La clé privée reste dans un service de secrets; la clé publique peut être distribuée aux vérificateurs. SHA-256 est calculé sur le fichier original et après reconstruction afin de vérifier l'identité bit-à-bit.

L'ordre des opérations suit encrypt-then-sign ou un schéma explicitement défini. Une ambiguïté sur la sérialisation peut invalider les signatures entre versions; la canonicalisation doit donc être documentée et testée.

4.1.6 Authentification JWT et autorisation RBAC

Le token JWT contient un identifiant utilisateur, un rôle, une date d'émission, une expiration et un identifiant de token. La validation contrôle l'algorithme attendu, la signature, l'expiration et les claims obligatoires. Les tokens expirés sont rejetés et les erreurs restent génériques afin de ne pas faciliter l'énumération.

RBAC associe les routes et opérations à des permissions. Le rôle user manipule ses fichiers; admin consulte les audits et gère les comptes. L'autorisation objet complète le RBAC en vérifiant que le file_id appartient au tenant actif. Ce contrôle empêche un utilisateur authentifié d'accéder aux objets d'un autre utilisateur.

Pour l'industrialisation, la révocation, le renouvellement et la rotation des clés de signature doivent être définis.

4.2.3.2 Captures d'écran et preuves d'exécution

Une capture académique doit être une preuve reproductible et non une illustration décorative. Elle comporte la version du logiciel, le scénario, la date et, si possible, l'identifiant du commit. Les écrans attendus couvrent le tableau de bord, le drag-and-drop, le résultat de compression, la décompression, l'historique, la quarantaine et la documentation Swagger.

Les captures réelles ne peuvent être produites à partir du mémoire sans accès à l'application exécutée. Le document conserve donc les emplacements et la méthodologie, mais ne présente aucune maquette comme une sortie réelle. Cette décision protège la sincérité scientifique du rapport.

Lors de l'intégration finale, les données sensibles, noms de fichiers internes et tokens doivent être masqués. Chaque capture doit être accompagnée d'une interprétation expliquant la fonctionnalité démontrée et le critère validé.

4.3.4 Système de logs et audit

Les logs applicatifs sont structurés et contiennent un timestamp, un niveau, un `trace_id`, un `user_id` pseudonymisé, un `tenant_id`, une action, une durée et un résultat. Les contenus des fichiers, tokens et clés ne sont jamais journalisés. Le `trace_id` relie la requête HTTP, la tâche Celery et les événements du scanner.

Les événements de sécurité sont séparés des logs de diagnostic. Ils documentent les décisions, règles déclenchées, hashes et actions de quarantaine. Le chaînage par hash rend les altérations détectables, mais ne remplace pas un stockage append-only avec contrôle d'accès.

Les politiques de rétention doivent respecter les besoins d'audit et la minimisation des données. Une intégration SIEM peut corréler les événements et produire des alertes.

4.3.5 Celery, Redis et Docker Compose

Celery exécute les tâches de compression batch en dehors du processus API. Les limites `soft` et `hard time limit` empêchent une tâche de monopoliser un worker. Les files peuvent être séparées selon la taille ou le type de traitement. Les résultats ont une durée de vie contrôlée pour éviter l'accumulation dans Redis.

Redis sert de broker et éventuellement de backend de résultats. Sa persistance, son authentification, son chiffrement réseau et sa capacité doivent être configurés selon l'environnement. Une indisponibilité ne doit pas empêcher les opérations synchrones qui n'en dépendent pas.

Docker Compose décrit les services, réseaux, volumes, healthchecks et variables. Les images sont construites de manière reproductible, exécutées en utilisateur non privilégié et analysées pour leurs vulnérabilités.

4.5.3 Méthodologie statistique des benchmarks

Chaque combinaison fichier-algorithme est exécutée plusieurs fois après échauffement. La médiane réduit l'effet des valeurs extrêmes, tandis que l'écart interquartile décrit la variabilité. La configuration CPU, mémoire, stockage, système et versions logicielles est figée et publiée.

Les caches peuvent être contrôlés selon l'objectif : cache chaud pour un service répétitif, cache froid pour mesurer un premier accès. Les temps incluent ou excluent explicitement les entrées-sorties. Les résultats de compression et décompression sont séparés, car les usages peuvent privilégier l'un ou l'autre.

L'interprétation doit éviter les conclusions globales à partir d'un seul corpus. Un gain sur texte ne justifie pas une supériorité sur les images ou archives.

4.5.4 Interprétation attendue des résultats

Une amélioration du ratio est pertinente seulement si elle dépasse l'overhead du conteneur et si le coût temporel reste compatible avec le cas d'usage. Pour les données fréquemment lues mais rarement écrites, une compression plus lente peut être acceptable. Pour les transferts interactifs, la latence devient prioritaire.

La mémoire et le CPU doivent être rapportés avec la taille des fichiers. Une croissance non linéaire peut révéler une implémentation BWT inadéquate. Le taux de fallback vers ZSTD direct permet d'évaluer la valeur réelle du pipeline transformé sur le corpus.

Les résultats négatifs sont scientifiquement utiles. Ils permettent de délimiter les familles de données pour lesquelles MOUVE apporte une valeur et d'orienter les optimisations futures.

4.6.1 Dataset IA et gouvernance des données

Le dataset est construit à partir de fichiers dont la provenance et la licence sont documentées. Les classes `TEXT`, `SOURCE_CODE`, `BINARY` et `RANDOM` doivent refléter la diversité réelle des formats. Les échantillons issus d'un

même fichier source ne sont pas répartis entre train et test, afin d'éviter que le modèle ne mémorise une signature spécifique.

Le nettoyage supprime les doublons exacts et quasi-doublons, vérifie les labels et contrôle les distributions de taille. Les données aléatoires sont générées par un mécanisme documenté et distinguées des données chiffrées ou compressées, dont les propriétés statistiques peuvent être proches.

La gouvernance inclut la version du dataset, le hash du manifeste, les scripts de préparation et les statistiques par classe.

4.6.2 Architecture et entraînement du classificateur

Le classificateur peut combiner des caractéristiques statistiques à un réseau compact sur histogrammes d'octets ou fenêtres. La couche de sortie softmax produit une distribution sur les classes. La loss d'entropie croisée est pondérée si le dataset reste déséquilibré. La régularisation et l'early stopping contrôlent le surapprentissage.

Les hyperparamètres comprennent le learning rate, l'optimiseur, la taille de batch, le nombre d'époques, la dimension des couches et le dropout. Une recherche limitée doit être effectuée sur la validation, jamais sur le test. La graine et le matériel sont enregistrés pour assurer la reproductibilité.

Le modèle est comparé à une baseline heuristique fondée sur l'entropie et le ratio imprimable. Son ajout n'est justifié que s'il améliore significativement la sélection sans introduire une latence excessive.

4.6.3 Métriques IA et analyse des erreurs

L'accuracy globale peut masquer une classe minoritaire. Le rapport doit présenter precision, recall et F1 par classe, F1 macro, matrice de confusion et intervalles de confiance. Les courbes de loss permettent de détecter le surapprentissage ou une optimisation instable.

L'analyse des erreurs examine les confusions entre binaire compressé et aléatoire, ou entre code source et texte. Ces erreurs ont un impact asymétrique : classer un fichier aléatoire comme texte peut déclencher un pipeline coûteux, tandis qu'un fallback vers ZSTD direct limite la dégradation fonctionnelle.

Une calibration des probabilités est recommandée si le score doit piloter une décision. À défaut, le score ne doit pas être présenté comme une probabilité fiable.

4.6.4 VQ-VAE, fonctions de perte et limites

Le VQ-VAE comprend un encodeur, une quantification vers un codebook et un décodeur. La loss combine reconstruction, engagement du codebook et mise à jour des embeddings. La taille du codebook et la dimension latente déterminent le compromis entre expressivité, taux et coût.

Pour un usage sans perte, la reconstruction approximative n'est pas suffisante. Un résidu exact pourrait être codé en complément, mais ce résidu peut annuler le gain. L'expérience doit donc mesurer la taille totale code latent, index, résidu, modèle et métadonnées.

Cette analyse peut conduire à repositionner le VQ-VAE comme outil de profilage ou de sélection plutôt que comme compresseur final. Une telle conclusion négative mais argumentée est pleinement acceptable dans un mémoire de recherche appliquée.

Chapitre 4. Conclusion

4.7 Analyse critique de la réalisation

La réalisation démontre la cohérence architecturale de MOUVE, mais la maturité industrielle dépend de résultats empiriques complets. Les points forts sont la modularité, la stratégie de repli ZSTD, la vérification d'intégrité, l'isolation tenant et la défense en profondeur. Les risques principaux sont le coût de BWT sur les grands blocs, la complexité opérationnelle du scanner, les faux positifs, la gestion des clés et l'absence éventuelle de bénéfice du mode appris sur des données non représentatives.

La conclusion technique doit donc être nuancée. MOUVE constitue une plateforme de recherche et d'ingénierie solide, mais la revendication d'un dépassement systématique de ZIP, GZIP, BZIP2 ou ZSTD doit être conditionnée

à une campagne expérimentale reproductible. Cette posture renforce la crédibilité académique du rapport et prépare des perspectives mesurables.

Ce chapitre a présenté la réalisation concrète de MOUVE. Les composants développés répondent aux objectifs définis dans les chapitres précédents : compression sans perte, sécurité des fichiers, interfaces multiples, conteneurs structurés et possibilité de déploiement. Les résultats obtenus constituent une base solide pour une industrialisation progressive et pour des travaux futurs centrés sur les benchmarks réels et l'amélioration du mode appris.

Conclusion générale

Ce rapport a présenté la conception et la réalisation de MOUVE, une plateforme intelligente de compression et de transfert sécurisé de fichiers. Le projet répond à une problématique actuelle : réduire la taille des données tout en garantissant leur intégrité, leur confidentialité et leur traçabilité. La solution proposée combine un pipeline classique BWT, MTF, RLE et ZSTD, une stratégie de sélection automatique, un mode appris optionnel, un scanner de sécurité et plusieurs interfaces d'utilisation.

Les objectifs annoncés dans l'introduction ont été structurés autour de quatre axes. Le premier concerne la compression sans perte et la décompression exacte. Le deuxième concerne la sécurité des fichiers entrants et des conteneurs produits. Le troisième concerne l'intégration multi-interface par API, CLI, Desktop et package Python. Le quatrième concerne la préparation au déploiement à travers Docker, Celery, Redis et une organisation modulaire.

Les limites du travail résident principalement dans la nécessité de réaliser des benchmarks sur des corpus réels plus larges, d'entraîner des modèles appris sur des données représentatives et de compléter l'interface Desktop par des captures et scénarios utilisateurs. Les perspectives portent sur l'automatisation de l'entraînement de dictionnaires, l'amélioration de l'IA de sélection, l'intégration cloud et la supervision détaillée des ratios de compression.

Perspectives scientifiques et industrialisation

L'industrialisation de MOUVE nécessite une campagne de performance reproductible, une revue de sécurité du format, une stratégie de gestion de clés, des migrations de conteneurs et une supervision opérationnelle. Le déploiement cloud peut exploiter un stockage objet, une file managée et une orchestration Kubernetes, mais il doit préserver l'isolation des tenants et le contrôle des coûts.

Les axes de recherche portent sur la sélection automatique de profils, l'entraînement de dictionnaires, le calcul distribué de BWT par blocs, la détection des données incompressibles et l'usage de modèles probabilistes réversibles. Une optimisation SIMD ou native peut améliorer les étapes MTF et RLE, tandis qu'une implémentation suffix array efficace réduit le coût de BWT.

La contribution future la plus crédible consiste à transformer MOUVE en banc d'essai extensible où chaque stratégie est comparée suivant un protocole commun. Cette orientation favorise la reproductibilité, la comparaison scientifique et le transfert vers des environnements industriels.

Références bibliographiques

Références complémentaires

- [21] OpenAPI Initiative, OpenAPI Specification 3.1.0, Linux Foundation, 2021.
- [22] Meta Platforms, React Documentation: Describing the UI and Managing State, 2026.
- [23] Vite Team, Vite Guide: Development Server and Production Build, 2026.
- [24] Celery Project, Celery Distributed Task Queue Documentation, 2026.
- [25] Celery Project, Using Redis as Broker and Result Backend, 2026.
- [26] A. van den Oord, O. Vinyals and K. Kavukcuoglu, “Neural Discrete Representation Learning,” NeurIPS, 2017.
- [27] Z. Duan et al., “QARV: Quantization-Aware ResNet VAE for Lossy Image Compression,” IEEE TPAMI, vol. 46, no. 1, pp. 436–450, 2024.
- [28] Docker Inc., Docker Documentation, 2026.
- [29] Cloud Native Computing Foundation, Kubernetes Documentation, 2026.
- [30] Redis Ltd., Redis Documentation, 2026.
- [31] ISO/IEC 25010:2023, Systems and software engineering — Product quality model, 2023.
- [32] ISO/IEC/IEEE 29119-1, Software and systems engineering — Software testing — Concepts and definitions, 2022.
- [33] NIST, Secure Software Development Framework (SSDF), SP 800-218, 2022.
- [34] OWASP Foundation, Web Security Testing Guide v4.2, Test Upload of Malicious Files, 2023.
- [35] Cisco Talos, ClamAV Documentation, 2026.
- [36] VirusTotal, YARA Documentation 4.5, 2026.
- [37] MITRE, CWE-434: Unrestricted Upload of File with Dangerous Type, 2026.
- [38] NIST, Digital Identity Guidelines, SP 800-63B, 2024.
- [39] IETF, JSON Web Token (JWT), RFC 7519, 2015.
- [40] IETF, OAuth 2.0 Authorization Framework, RFC 6749, 2012.
- [41] A. D. Birrell and B. J. Nelson, “Implementing Remote Procedure Calls,” ACM TOCS, 1984.
- [42] R. C. Martin, Clean Architecture: A Craftsman’s Guide to Software Structure and Design, Prentice Hall, 2017.
- [43] E. Gamma et al., Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.
- [44] M. Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, 2002.
- [45] The PostgreSQL Global Development Group, PostgreSQL Documentation, 2026.
- [1] M. Burrows and D. J. Wheeler, “A Block-sorting Lossless Data Compression Algorithm,” Digital Equipment Corporation, Systems Research Center, Research Report 124, 1994.
- [2] G. Manzini, “The Burrows-Wheeler Transform: Theory and Practice,” in Mathematical Foundations of Computer Science, 1999.
- [3] Y. Collet and M. Kucherawy, “Zstandard Compression and the application/zstd Media Type,” RFC 8878, IETF, 2021.
- [4] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, 2015.
- [5] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” NIST SP 800-38D, 2007.
- [6] S. Josefsson and I. Liusvaara, “Edwards-Curve Digital Signature Algorithm (EdDSA),” RFC 8032, IRTF, 2017.
- [7] OWASP Foundation, “File Upload Cheat Sheet,” OWASP Cheat Sheet Series, 2026.
- [8] S. Ramírez, “FastAPI Documentation,” FastAPI Project, 2026.
- [9] OpenAPI Initiative, “OpenAPI Specification, Version 3.1.0,” Linux Foundation, 2021.

- [10] PyTorch Foundation, "PyTorch Documentation," Linux Foundation, 2026.
- [11] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural Discrete Representation Learning," Advances in Neural Information Processing Systems, 2017.
- [12] Celery Project, "Celery: Distributed Task Queue Documentation," 2026.
- [13] Docker Inc., "Docker Documentation," 2026.
- [14] Vite Team, "Vite Guide," 2026.
- [15] K. Schwaber and J. Sutherland, "The Scrum Guide: The Definitive Guide to Scrum," 2020.
- [16] AICPA, "Trust Services Criteria for Security, Availability, Processing Integrity, Confidentiality, and Privacy," 2017, updated 2022.
- [17] AICPA & CIMA, "System and Organization Controls: SOC Suite of Services," 2026.
- [18] J. Ziv and A. Lempel, "A Universal Algorithm for Sequential Data Compression," IEEE Transactions on Information Theory, 1977.
- [19] D. A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes," Proceedings of the IRE, 1952.
- [20] C. E. Shannon, "A Mathematical Theory of Communication," Bell System Technical Journal, 1948.

Références additionnelles

- [46] P. Deutsch, “DEFLATE Compressed Data Format Specification,” RFC 1951, IETF, 1996.
- [47] P. Deutsch, “GZIP File Format Specification Version 4.3,” RFC 1952, IETF, 1996.
- [48] J. Seward, “bzip2 and libbzip2 Documentation,” 2019.
- [49] I. Pavlov, “LZMA SDK Documentation,” 7-Zip Project, 2024.
- [50] NIST, “Guide to Storage Encryption Technologies for End User Devices,” SP 800-111, 2007.
- [51] Microsoft, “STRIDE Threat Modeling,” Security Engineering Documentation, 2025.
- [52] MITRE, “CWE-22: Improper Limitation of a Pathname to a Restricted Directory,” 2026.
- [53] IETF, “The OAuth 2.0 Authorization Framework: Bearer Token Usage,” RFC 6750, 2012.
- [54] ISO/IEC 27001:2022, Information security management systems — Requirements, 2022.
- [55] D. Merkel, “Docker: Lightweight Linux Containers for Consistent Development and Deployment,” Linux Journal, 2014.

Annexes

Annexe 1. Exemple de configuration API

Note sur les captures d'écran réelles

Les captures d'écran réelles de l'interface, de Swagger UI, de la CLI et des résultats de benchmark ne peuvent pas être déduites du rapport seul. Afin de préserver l'intégrité académique, aucune maquette n'est présentée comme une preuve d'exécution. Les emplacements correspondants doivent être remplacés par des captures provenant de l'instance MOUVE réellement exécutée, avec date, version du commit, environnement et scénario reproduit.

Les captures attendues sont : écran d'accueil Desktop, workflow de compression, résultat de décompression, historique, documentation Swagger, sortie CLI, statut d'un job Celery, journaux de sécurité et dashboard de benchmark. Chaque capture doit être accompagnée d'une légende analysant ce qu'elle démontre.

Cette annexe présente un exemple de configuration destinée à l'exécution locale de l'API MOUVE. Les paramètres réels doivent être adaptés à l'environnement de déploiement, notamment pour les secrets, les chemins de stockage et les politiques d'authentification.

```
MOUVE_DEV_NO_AUTH=1
MOUVE_STORAGE_ROOT=./data/tenants
MOUVE_SECURITY_LOGS=./logs/security.jsonl
MOUVE_ZSTD_LEVEL=9
MOUVE_ENABLE_NVC=1
```

Annexes

Annexe 2. Entreprise et résumé final

Cette annexe peut accueillir le logo de l'entreprise d'accueil, les coordonnées administratives et le résumé final du rapport. Le résumé doit rester synthétique et présenter le contexte, la solution, les technologies utilisées et les principaux apports du projet.

Résumé : MOUVE est une plateforme intelligente de compression et de transfert sécurisé de fichiers. Elle combine une chaîne de compression hybride, un scanner de sécurité multicouche, des conteneurs vérifiables et quatre interfaces d'utilisation. Le projet démontre l'intérêt d'une approche intégrée associant performance, sécurité, auditabilité et facilité d'intégration.

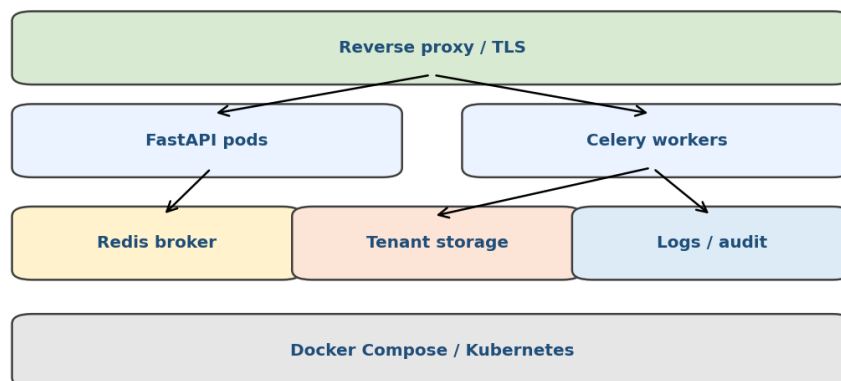


Figure annexe 2.1 : Déploiement cible de MOUVE